

WorkBuddy AI 教程

从 Prompt 到交付：AI Agent 工作方法论

版本：v1.0 更新日期：2026 年 5 月 适用人群：已经用过 AI、想用出生产力的知识工作者
阅读时长：约 3 小时

前言

如果你已经用过 ChatGPT、文心一言、通义千问，却发现它们只能“聊天”而无法真正“干活”，那这本文档就是为你准备的。

2026 年，AI 工具的分水岭不再是“聪明程度”，而是能否交付结果。WorkBuddy 代表的 AI Agent 范式，把“问答式 AI”升级为“任务式 AI”——你不再需要手把手教它每一步，只需要说清楚目标、给足素材、定好验收标准，它会自己规划、执行、验证。

这本文档不是产品说明书，而是工作方法论。你会学到：

- * 为什么 Agent 不是更聪明的聊天机器人
- * 怎么把自己从“操作者”变成“管理者”
- * 如何写出能交付结果的 Brief（而不是 Prompt）
- * 5 个跨职能场景的完整复盘（市场/销售/财务/产品/研发）
- * 30 条可直接复用的 Brief 模板

阅读建议：

- * 如果你只想快速上手：读第 1-3 章 + 第 8 章任意一个场景
- * 如果你想用熟：全部读完，重点看第 8 章
- * 如果你想做组织级推广：重点看第 9 章

完整目录

● 第 1 章 为什么 Agent 不是更聪明的 ChatBot

- * 从"问答"到"交付": 范式差别
- * Agent 三件套: 规划、工具、记忆
- * WorkBuddy 在 Agent 光谱里的位置
- * 一张 ROI 判断表: 什么任务值得交给 Agent

● 第 2 章 心法: 把自己从操作者变成管理者

- * 别再手把手教, 学会"交付"
- * 四件套: 目标 / 约束 / 素材 / 验收
- * 任务粒度: 一次该交多大活
- * "先列计划再动手"是最值钱的一句口令

● 第 3 章 指令工程: 从 Prompt 到 Brief

- * Prompt 和 Brief 的差别
- * 三段式 Brief 模板
- * 为什么"写个周报"永远得到废话
- * 让 Agent 反问你: 苏格拉底式澄清
- * 把一次性 Prompt 沉淀为模板

● 第 4 章 读懂它在干嘛: Agent 执行链路

- * 计划--执行--验证的闭环
- * 看懂任务树: 哪一步出了问题
- * 中断 / 纠偏 / 回滚的正确姿势
- * 判断"让它继续跑"还是"现在就拽回来"

● 第 5 章 沙箱与工具调用: 能力边界

- * 沙箱是安全网, 不是限制
- * 本地文件 / 浏览器 / 外部 API 的边界
- * 给敏感数据画围栏
- * 不可逆操作的双重确认

● 第 6 章 技能包与 MCP：让 Agent 长出手脚

- * 20+ 预置技能包：怎么选、怎么组合
- * MCP 的本质：给 Agent 装插件
- * 接入企业系统的三种路径
- * 什么时候自己写、什么时候别碰

● 第 7 章 workflow 设计：一次性 -> 可复用 -> 自动化

- * 三种任务形态的识别
- * 把“每周都要做”固化为流程
- * 多 Agent 协同：让专业的人做专业的事
- * 触发器、调度、异常处理

● 第 8 章 跨职能场景深度解剖

- * 8.1 竞品动态周报（市场）
- * 8.2 销售线索打分与跟进（销售）
- * 8.3 月度对账与异常标注（财务）
- * 8.4 用户反馈主题洞察（产品）
- * 8.5 技术选型调研（研发）

● 第 9 章 协作模式：从个人到组织

- * 个人：建立自己的 Brief 库和技能库
- * 团队：共享流程和技能包的治理
- * 组织：权限、审计、合规

● 第 10 章 判断力：什么时候不该用 Agent

- * Agent 幻觉比 ChatBot 更危险
- * 验收 vs 信任：永远的人机边界
- * 成本意识：token、时间、决策质量三本账
- * 6 类任务真的不划算

● 附录

- * A. 30 条 Brief 模板（按职能分类）
- * B. MCP 生态地图与接入清单
- * C. 常见故障诊断树
- * D. 从入门到熟练的 30 天学习路径

第 1 章 为什么 Agent 不是更聪明的 ChatBot

● 1.1 从"问答"到"交付": 范式差别

当你打开 ChatGPT 问"怎么写周报", 它会给你一个模板、几条建议, 然后等你自己动手。这是问答范式: AI 提供信息, 人类执行操作。

当你向 WorkBuddy 说"帮我写本周的竞品周报, 素材在桌面的 research 文件夹, 输出格式参考上周的模板", 它会:

- 1. 读取文件夹里的所有素材
- 2. 分析竞品动态的关键信息
- 3. 按照模板结构组织内容
- 4. 生成 Word 文档并保存到指定位置
- 5. 向你汇报完成情况

这是交付范式: AI 完成整个任务链条, 人类只需验收结果。

核心差别不是"更聪明", 而是"能动手"。

维度	ChatBot（问答式）	Agent（任务式）
交互方式	一问一答	目标导向
工作单元	单轮对话	多步任务
工具能力	无（纯文本输出）	有（文件、浏览器、API）
自主性	被动响应	主动规划
验收标准	回答质量	交付结果

1.2 Agent 三件套：规划、工具、记忆

一个合格的 AI Agent 必须具备三项核心能力，缺一不可。

● 规划 (Planning)

Agent 收到任务后，第一件事不是立刻动手，而是分解任务、排列顺序、预判风险。

举例：你说“帮我整理桌面文件”，Agent 的规划过程：

1. 扫描桌面文件列表
2. 按文件类型分类（文档/图片/压缩包/其他）
3. 创建对应文件夹
4. 移动文件到对应文件夹
5. 生成整理报告

这个“先列计划再动手”的能力，是 Agent 和 ChatBot 的第一道分水岭。没有规划能力的 AI，只能做单步操作；有规划能力的 AI，才能完成复杂任务。

● 工具 (Tools)

光有计划不够，还得有“手脚”去执行。Agent 的工具箱通常包括：

- * 文件操作：读、写、移动、重命名、格式转换
- * 浏览器：搜索、抓取网页、填表单、截图
- * 数据处理：Excel 计算、数据清洗、图表生成
- * 外部 API：调用企业系统、数据库、第三方服务

WorkBuddy 预置了 20+ 技能包，每个技能包就是一组工具的集合。比如“竞品监控”技能包包含：网页抓取 + 关键词提取 + 对比分析 + 报告生成。

● 记忆 (Memory)

Agent 需要记住三类信息：

1. 任务上下文：这次任务的目标、素材、中间结果
2. 用户偏好：你习惯的输出格式、常用的文件路径、行业术语
3. 历史经验：上次类似任务怎么做的、哪里出过错

没有记忆的 Agent，每次都是“初次见面”，你得重复交代背景；有记忆的 Agent，会越用越懂你。

1.3 WorkBuddy 在 Agent 光谱里的位置

2026 年的 Agent 市场已经分化出明确的赛道，不同产品服务不同场景。

产品	定位	核心场景	部署方式	扩展性
Cursor	代码编辑器 Agent	写代码、改 bug、重构	本地 IDE	中（插件）
Manus	通用桌面 Agent	跨应用操作、UI 自动化	本地桌面	高（脚本）
Dify	Agent 开发平台	自建企业 Agent	云端/私有化	极高（低代码）
WorkBuddy	办公场景 Agent	文档、数据、调研类任务	本地桌面	高（MCP）

WorkBuddy 的甜点区域：

- * [完成] 文档密集型工作（周报、调研、会议纪要）
- * [完成] 数据处理任务（Excel、对账、清洗）
- * [完成] 信息获取任务（竞品监控、资讯汇总）
- * [错误] 代码开发（用 Cursor 更合适）
- * [错误] 复杂 UI 自动化（用 Manus 更合适）
- * [错误] 定制化企业流程（用 Dify 更合适）

1.4 一张 ROI 判断表：什么任务值得交给 Agent

不是所有任务都适合交给 Agent。判断标准不是“能不能做”，而是“值不值得做”。

● 高 ROI 任务特征（优先交给 Agent）

特征	说明	举例
重复性高	每周/每月都要做	周报、月度对账、竞品监控
规则明确	步骤可枚举、判断标准清晰	发票分类、简历初筛、数据清洗
信息密集	需要处理大量文档/网页	行业调研、用户反馈汇总
格式转换	从 A 格式整理成 B 格式	PDF 提取表格、聊天记录生成纪要
时间敏感	人工做太慢，Agent 能加速	舆情监控、实时数据汇总

● 低 ROI 任务特征（不建议交给 Agent）

特征	说明	为什么不适合
一次性任务	只做一次，不会重复	配置 Agent 的时间 > 直接做的时间
高度创意	需要灵感、审美、情感	Agent 输出平庸，还得大改
强依赖判断	每一步都需要人类决策	来回确认比自己做更慢
数据敏感	涉及机密、隐私、合规	风险大于收益
工具链复杂	需要 10+ 个系统协同	接入成本太高

快速判断公式

1 Agent ROI = (节省时间 × 重复次数 - 配置成本) / 风险系数

- * 节省时间：Agent 做 vs 人工做的时间差
- * 重复次数：未来 3 个月会做几次
- * 配置成本：写 Brief、调试、验收的时间
- * 风险系数：出错后的影响（1-10 分）

举例：

- * 每周竞品周报：节省 2 小时 × 12 次 - 1 小时配置 = 23 小时，风险低（3 分）-> 高 ROI
- * 年度战略 PPT：节省 5 小时 × 1 次 - 2 小时配置 = 3 小时，风险高（8 分）-> 低 ROI

一个反直觉的结论

很多人第一次用 Agent，会想“让它帮我做最难的事”。这是错的。

Agent 的价值不是替你做难事，而是替你做烦事。

难事往往需要创意、判断、灵活应变，这些恰恰是 Agent 的弱项。烦事（重复、规则明确、信息密集）才是 Agent 的主场。

把 Agent 当成“初级助理”，不是“高级专家”，你的期待就对了。

本章小结

- * Agent 和 ChatBot 的本质差别是“交付”而不是“聊天”
- * Agent 三件套：规划（分解任务）、工具（执行操作）、记忆（积累经验）
- * WorkBuddy 适合文档、数据、调研类办公任务，不适合代码开发和复杂 UI 自动化
- * 高 ROI 任务特征：重复性高、规则明确、信息密集、格式转换、时间敏感

* Agent 的价值是替你做烦事，不是替你做难事

第 2 章 心法：把自己从操作者变成管理者

2.1 别再手把手教，学会“交付”

大部分人第一次用 Agent，会这样说话：

> “先打开桌面的 research 文件夹，然后找到所有 PDF 文件，把每个 PDF 的第一页内容提取出来，再整理成一个 Excel 表格，第一列是文件名，第二列是提取的内容……”

这是操作者思维：把自己会怎么做的步骤，一步步教给 AI。

问题在于：

1. 你说的步骤可能不是最优路径
2. 中间某一步出错，整个流程卡住
3. 你花了 10 分钟写指令，Agent 花了 2 分钟执行，总时间没省多少

换成管理者思维，应该这样说：

> “把桌面 research 文件夹里所有 PDF 的首页内容提取出来，整理成 Excel 表格，两列：文件名和内容摘要。如果某个 PDF 打不开或者首页是空白，跳过并在备注里标注。”

区别在哪？

- * 你说的是目标（提取内容、整理表格），不是步骤（先打开、再提取）
- * 你给了约束（两列、跳过异常），不是流程（怎么处理异常）
- * 你留了自主空间，让 Agent 自己选最优路径

核心转变：从“教它怎么做”到“告诉它要什么”。

2.2 四件套：目标 / 约束 / 素材 / 验收

一个合格的任务交付，必须包含四个要素。缺任何一个，Agent 都会“理解偏差”或“输出不符合预期”。

● 目标（What）

要什么结果，用一句话说清楚。

[错误] 坏例子: "帮我处理一下这些文件" [完成] 好例子: "把这些发票按供应商分类, 生成汇总表"

目标要具体到"可验收"的程度。"处理一下"无法验收, "生成汇总表"可以验收。

● 约束 (Constraints)

不能做什么、必须遵守什么规则。

常见约束类型:

- * 格式约束: "输出必须是 Word 文档, 不要 PDF"
- * 范围约束: "只看最近 30 天的数据"
- * 质量约束: "每个竞品至少 3 条动态, 少于 3 条的不要"
- * 安全约束: "不要动 archive 文件夹里的文件"

约束是"防护栏", 防止 Agent 跑偏。没有约束的任务, Agent 会按自己的理解做, 结果往往不是你要的。

● 素材 (Materials)

Agent 需要哪些输入。

- * 文件路径: "桌面的 research 文件夹"
- * 网址: "从 36kr.com 抓取本周融资新闻"
- * 参考模板: "参考上周的周报格式"
- * 背景信息: "我们公司是做 SaaS 的, 竞品是 A、B、C 三家"

素材要"可定位"。"那些文件"不如"桌面 research 文件夹", "类似的格式"不如"参考 report_template.docx"。

● 验收 (Acceptance)

怎么判断任务完成了。

- * 交付物: "生成一个 Excel 文件, 保存到桌面"
- * 质量标准: "每个竞品至少 3 条动态"
- * 异常处理: "如果某个网站打不开, 在备注里说明"

验收标准是"任务完成的定义"。没有验收标准, Agent 不知道什么时候该停下来。

● 四件套模板

- 1 【目标】把桌面 research 文件夹里所有 PDF 的首页内容提取出来，整理成 Excel 表格
- 2 【约束】只提取首页，跳过加密或损坏的 PDF
- 3 【素材】桌面 research 文件夹（约 50 个 PDF）
- 4 【验收】生成 research_summary.xlsx，两列：文件名、首页摘要（100 字以内）

2.3 任务粒度：一次该交多大活

任务太小，来回沟通成本高；任务太大，Agent 容易跑偏。

● 三种粒度

粒度	时长	适合场景	举例
原子任务	1-5 分钟	单步操作	"把这个 PDF 转成 Word"
组合任务	10-30 分钟	多步流程	"整理桌面文件并生成清单"
项目任务	1-3 小时	复杂目标	"完成本周竞品周报"

推荐粒度：组合任务。

- * 原子任务太碎，你会变成"任务调度员"，一直在下指令
- * 项目任务太大，Agent 容易在某一步卡住，你不知道它在干嘛

组合任务是甜点：Agent 有足够自主空间，你有足够掌控感。

● 拆分原则

如果一个任务符合以下任一条件，建议拆分：

- 1. 需要人类决策：比如"根据调研结果决定是否继续"
- 2. 依赖外部输入：比如"等老板审批后再继续"
- 3. 风险不可控：比如"删除所有重复文件"（万一误删？）

拆分后的任务，用"先做 A，完成后我再告诉你做 B"的方式串联。

2.4 "先列计划再动手"是最值钱的一句口令

Agent 最容易犯的错误：收到任务就开始执行，结果方向错了，白干。

解决方案：强制规划。

在任务开头加一句：

＞ "先列出详细计划，包括每一步要做什么、预计耗时、可能的风险，等我确认后再执行。"

这句话的价值：

1. 暴露理解偏差：Agent 列出的计划和你预期不一致，说明它理解错了
2. 预判风险：Agent 会标注"这一步可能失败"，你可以提前调整
3. 建立信任：你看到计划合理，才会放心让它执行

● 计划的标准格式

```
1  【任务】整理桌面文件并生成清单
2
3  【计划】
4  1. 扫描桌面文件列表（预计 10 秒）
5  2. 按文件类型分类：文档/图片/压缩包/其他（预计 30 秒）
6  3. 创建对应文件夹（预计 10 秒）
7  4. 移动文件到对应文件夹（预计 1 分钟，风险：可能有文件正在使用）
8  5. 生成清单 Excel：文件名、类型、大小、修改时间（预计 30 秒）
9  6. 保存到桌面 desktop_cleanup_report.xlsx（预计 5 秒）
10
11 【风险】
12 - 如果有文件正在使用，移动会失败，我会跳过并在清单里标注
13 - 如果桌面文件超过 1000 个，可能需要 5-10 分钟
14
15 【确认】以上计划是否 OK？
```

看到这个计划，你可以：

- * 确认方向对不对（比如你其实不想移动文件，只想生成清单）
- * 调整细节（比如"压缩包不要动，放在原地"）
- * 评估风险（比如"桌面确实有 2000 个文件，可能很慢"）

这一步多花 1 分钟，能省后面 10 分钟返工。

本章小结

- * 从操作者思维切换到管理者思维：说目标，不说步骤
- * 任务交付四件套：目标（要什么）、约束（不能做什么）、素材（输入是什么）、验收（怎么算完成）
- * 推荐任务粒度：组合任务（10-30 分钟），不要太碎也不要太大
- * 强制规划："先列计划再动手"，暴露理解偏差、预判风险、建立信任

第 3 章 指令工程：从 Prompt 到 Brief

3.1 Prompt 和 Brief 的差别

大部分人用 ChatGPT 时写的是 Prompt（提示词），用 Agent 时需要写的是 Brief（任务简报）。两者看起来都是“给 AI 的指令”，但本质不同。

维度	Prompt（提示词）	Brief（任务简报）
目标	获得一个回答	交付一个结果
长度	1-3 句话	3-10 句话
结构	随意	结构化（目标/约束/素材/验收）
迭代	一次性	可复用、可模板化
验收	主观（回答好不好）	客观（结果对不对）

举例对比：

Prompt 风格（适合 ChatGPT）： > "帮我写一份竞品周报"

Brief 风格（适合 WorkBuddy）： > 【目标】生成本周竞品动态周报 > 【约束】只看 A、B、C 三家竞品，每家至少 3 条动态，按时间倒序排列 > 【素材】从 36kr、虎嗅、公司官网抓取本周（5月1-7日）的新闻 > 【验收】生成 Word 文档，包含：竞品名称、动态标题、来源链接、一句话总结

Brief 的核心是可验收。你能明确判断"做完了"还是"没做完"，"做对了"还是"做错了"。

3.2 三段式 Brief 模板

一个标准的 Brief 分三段：背景 + 任务 + 交付。

● 第一段：背景（Context）

为什么要做这件事，Agent 需要知道什么背景信息。

背景不是废话，而是帮助 Agent 做判断的依据。

[错误] 没有背景： > "帮我筛选简历"

[完成] 有背景： > "我们在招 Python 后端工程师，要求 3 年以上经验、熟悉 Django/Flask、有大厂背景优先。帮我从这 50 份简历里筛选出符合条件的。"

有了背景，Agent 才知道"3 年以上经验"是硬性要求，"大厂背景"是加分项。

● 第二段：任务（Task）

具体要做什么，用四件套（目标/约束/素材/验收）说清楚。

- 1 【目标】筛选出符合条件的简历
- 2 【约束】
- 3 - 必须满足：Python 3年+、熟悉 Django 或 Flask
- 4 - 加分项：大厂背景、有开源项目
- 5 - 排除：只会 Python 爬虫、没有 Web 开发经验
- 6 【素材】桌面 resumes 文件夹（50 份 PDF）
- 7 【验收】生成 Excel 表格，列出：姓名、工作年限、技术栈、匹配度（高/中/低）、推荐理由

● 第三段：交付（Delivery）

最终输出是什么，放在哪里，什么格式。

- 1 【交付】
- 2 - 文件名：resume_shortlist_2026-05-08.xlsx
- 3 - 保存位置：桌面
- 4 - 格式：Excel，包含筛选结果和原始简历的文件名映射
- 5 - 附加：如果有特别优秀的候选人，单独标注"强烈推荐"

● 完整示例

- 1 【背景】
- 2 我们在招 Python 后端工程师，要求 3 年以上经验、熟悉 Django/Flask、有大厂背景优先。
- 3
- 4 【任务】
- 5 目标：筛选出符合条件的简历
- 6 约束：
- 7 - 必须满足：Python 3年+、熟悉 Django 或 Flask
- 8 - 加分项：大厂背景、有开源项目
- 9 - 排除：只会 Python 爬虫、没有 Web 开发经验
- 10 素材：桌面 resumes 文件夹（50 份 PDF）
- 11 验收：生成 Excel 表格，列出：姓名、工作年限、技术栈、匹配度（高/中/低）、推荐理由
- 12
- 13 【交付】
- 14 - 文件名：resume_shortlist_2026-05-08.xlsx
- 15 - 保存位置：桌面
- 16 - 格式：Excel，包含筛选结果和原始简历的文件名映射
- 17 - 附加：如果有特别优秀的候选人，单独标注"强烈推荐"

3.3 为什么"写个周报"永远得到废话

这是最常见的失败案例：

> "帮我写个周报"

Agent 输出：

> 本周工作总结：> 1. 完成了项目 A 的开发 > 2. 参加了 3 次会议 > 3. 解决了 5 个 bug > >
下周计划：> 1. 继续推进项目 B > 2. 优化系统性能

这种周报的问题：全是废话，没有信息量。

为什么会这样？因为你没给 Agent 足够的信息：

- * 周报给谁看？（老板 / 团队 / 客户，关注点不同）
- * 周报要体现什么？（进度 / 成果 / 风险）
- * 有哪些素材？（会议纪要 / 代码提交 / 聊天记录）

改进版 Brief：

- 1 【背景】
- 2 我是产品经理，周报给老板看，老板关心：项目进度、关键决策、风险预警。
- 3
- 4 【任务】
- 5 目标：生成本周工作周报
- 6 约束：
- 7 - 重点写"做了什么决策、为什么这么决策"，不要流水账
- 8 - 风险部分必须有，哪怕没有风险也要写"本周无重大风险"
- 9 - 篇幅控制在 500 字以内
- 10 素材：
- 11 - 本周会议纪要（meeting_notes 文件夹）
- 12 - 项目进度表（project_tracker.xlsx）
- 13 - 我和团队的聊天记录（可以问我要）
- 14 验收：生成 Word 文档，包含三部分：本周进展、关键决策、下周计划 + 风险
- 15
- 16 【交付】
- 17 - 文件名：weekly_report_2026-W19.docx
- 18 - 保存位置：桌面
- 19 - 格式：Word，使用公司周报模板（参考 report_template.docx）

这样写出来的周报，才有信息量。

3.4 让 Agent 反问你：苏格拉底式澄清

有时候你自己也不确定怎么描述任务，这时候可以让 Agent 帮你澄清。

技巧：在 Brief 结尾加一句“如果有不清楚的，先列出问题问我”。

举例：

你说：> “帮我做一份竞品分析”

Agent 反问：> 在开始之前，我需要确认几个问题：> 1. 竞品是哪几家？（如果不确定，我可以帮你找）> 2. 分析维度是什么？（产品功能 / 价格策略 / 市场定位 / 用户评价）> 3. 分析深度？（简单对比 / 深度拆解）> 4. 输出格式？（PPT / Word / Excel）> 5. 给谁看？（老板 / 团队 / 投资人，关注点不同）

你回答：> 1. 竞品是 A、B、C 三家 > 2. 重点看产品功能和价格策略 > 3. 简单对比就行，不用太深 > 4. PPT，给老板看 > 5. 老板关心“我们和竞品的差异在哪”

Agent 确认：> 明白了，我会：> 1. 抓取 A、B、C 三家的官网和产品介绍 > 2. 对比产品功能（列表形式）和价格策略（表格形式）> 3. 生成 PPT，重点突出“我们 vs 竞品的差异”> 4. 保存到桌面，文件名 competitor_analysis_2026-05-08.pptx >> 开始执行？

这种“反问 -> 确认 -> 执行”的流程，比你一开始就写一个不完整的 Brief 要高效得多。

3.5 把一次性 Prompt 沉淀为模板

如果一个任务每周/每月都要做，别每次都重新写 Brief，把它沉淀为模板。

模板化三步骤：

● 第 1 步：识别变量

把 Brief 里会变的部分标记出来。

原始 Brief：> 生成本周竞品动态周报，素材从 36kr 抓取 5月1-7日 的新闻

模板化：> 生成本周竞品动态周报，素材从 36kr 抓取 {{日期范围}} 的新闻

● 第 2 步：固化结构

把不变的部分写死。

- 1 【背景】
- 2 我是市场部，每周一需要给老板汇报竞品动态。
- 3
- 4 【任务】
- 5 目标：生成本周竞品动态周报
- 6 约束：只看 A、B、C 三家竞品，每家至少 3 条动态
- 7 素材：从 36kr、虎嗅抓取 {{日期范围}} 的新闻
- 8 验收：生成 Word 文档，包含：竞品名称、动态标题、来源链接、一句话总结
- 9
- 10 【交付】
- 11 - 文件名：competitor_weekly_{{日期}}.docx
- 12 - 保存位置：桌面 reports 文件夹
- 13 - 格式：Word，使用 report_template.docx 模板

● 第 3 步：一键调用

下次用的时候，只需要填变量：

➢ 用“竞品周报模板”，日期范围 5月8-14日

Agent 自动替换变量，执行任务。

模板的价值：

- * 节省时间：从 5 分钟写 Brief 降到 10 秒填变量
- * 减少错误：结构固定，不会漏掉关键信息
- * 可传承：新人接手，直接用模板，不用重新摸索

本章小结

- * Prompt 是“获得回答”，Brief 是“交付结果”，后者需要结构化（目标/约束/素材/验收）
- * 三段式 Brief：背景（为什么做）+ 任务（怎么做）+ 交付（输出什么）
- * “写个周报”得到废话，是因为没给足背景和约束
- * 让 Agent 反问你，用苏格拉底式澄清补全信息
- * 重复任务要模板化，识别变量、固化结构、一键调用

第 4 章 读懂它在干嘛：Agent 执行链路

● 4.1 计划--执行--验证的闭环

Agent 收到任务后，不是立刻开始干活，而是走一个标准流程：

1. 理解任务 → 2. 制定计划 → 3. 执行步骤 → 4. 验证结果 → 5. 汇报交付

这个闭环是 Agent 和传统脚本的本质区别。脚本是“写死的流程”，Agent 是“自适应的流程”。

● 阶段 1：理解任务

Agent 会解析你的 Brief，提取关键信息：

- * 目标是什么
- * 约束有哪些
- * 素材在哪里
- * 验收标准是什么

如果信息不足，Agent 会反问（如果你在 Brief 里写了“有疑问先问我”）。

● 阶段 2：制定计划

Agent 把任务分解成多个子步骤，每个子步骤包含：

- * 要做什么操作
- * 需要调用什么工具
- * 预计耗时
- * 可能的风险

这个计划会展示给你确认（如果你要求“先列计划”）。

● 阶段 3：执行步骤

Agent 按计划逐步执行，每一步会：

- * 调用对应的工具（文件读写、浏览器、API）
- * 记录中间结果
- * 判断是否成功

如果某一步失败，Agent 会尝试：

1. 重试（比如网络超时）
2. 换方案（比如网站打不开，换个数据源）
3. 跳过并标注（比如某个文件损坏）

● 阶段 4：验证结果

执行完成后，Agent 会对照验收标准自查：

- * 交付物生成了吗
- * 格式对不对
- * 内容完整吗
- * 有没有遗漏的异常

● 阶段 5：汇报交付

Agent 向你汇报：

- * 任务完成情况
- * 生成了什么文件
- * 有没有异常或需要注意的地方

● 4.2 看懂任务树：哪一步出了问题

Agent 执行任务时，会生成一个“任务树”，展示每一步的状态。

示例任务树：

- 1 生成竞品周报
- 2 └─ 1. 抓取 36kr 本周新闻（耗时 15s）
- 3 └─ 2. 抓取虎嗅本周新闻（耗时 12s）
- 4 └─ 3. 抓取竞品 A 官网（失败：网站无法访问）
- 5 └─ 4. 提取关键信息（耗时 8s）
- 6 └─ 5. 按模板生成 Word（耗时 5s）
- 7 └─ 6. 保存到桌面（耗时 1s）
- 8
- 9 警告：竞品 A 官网无法访问，已跳过，建议手动补充

看懂任务树的关键：

符号	含义	你该做什么
[完成]	成功	不用管
[警告]	警告（部分失败）	看看影响大不大，决定是否手动补救
[错误]	失败（任务中断）	看失败原因，调整 Brief 或素材后重试

☐	等待确认	Agent 在等你的输入，赶紧回复
--------------------------	------	-------------------

常见失败原因：

- 1. 素材问题：文件路径错误、文件损坏、权限不足
- 2. 网络问题：网站打不开、API 超时、被反爬
- 3. 理解偏差：Agent 理解错了你的意图
- 4. 工具限制：某个操作超出了 Agent 的能力范围

4.3 中断 / 纠偏 / 回滚的正确姿势

Agent 执行任务时，你不是“旁观者”，而是“监工”。发现问题要及时介入。

● 中断（Stop）

什么时候中断：

- * Agent 理解错了，方向不对
- * 某一步卡住了，一直在重试
- * 你突然想起来有更重要的事

怎么中断：直接说“停下来”或点击“停止”按钮。Agent 会立刻停止，保留已完成的部分。

中断后怎么办：

- * 如果是理解偏差：重新写 Brief，说清楚哪里理解错了
- * 如果是卡住了：看任务树，找到卡住的那一步，调整素材或约束后重试
- * 如果是临时有事：说“先暂停，我一会儿回来继续”

● 纠偏（Correct）

什么时候纠偏：

- * Agent 做对了 80%，但某个细节不对
- * 你想调整某一步的输出

怎么纠偏：不要说“重新做”，而是说“在当前基础上调整”。

[错误] 低效纠偏：> “不对，重新做一遍，这次要……”

[完成] 高效纠偏：> “第 3 步的竞品 A 数据不对，改成从他们的公众号抓取，其他步骤不变”

Agent 会只重做第 3 步及后续受影响的步骤，不会从头开始。

- 回滚 (Rollback)

什么时候回滚：

- * Agent 做了不可逆操作（比如删除文件、发送邮件）
- * 你发现结果完全不对，想恢复到执行前的状态

怎么回滚：WorkBuddy 的沙箱机制会自动备份关键操作。你可以说：

> "撤销刚才的操作，恢复到执行前的状态"

但注意：不是所有操作都能回滚。比如：

- * [完成] 可回滚：文件修改、文件移动
- * [错误] 不可回滚：发送邮件、调用外部 API、删除云端数据

所以高风险操作前，一定要让 Agent 先列计划，你确认后再执行。

4.4 判断"让它继续跑"还是"现在就拽回来"

Agent 执行任务时，你会面临一个选择：是让它继续跑，还是现在就中断？

- 让它继续跑的情况

- * [完成] 方向对，只是某一步慢一点（比如网络慢）
- * [完成] 出现警告，但不影响核心目标（比如某个竞品数据缺失）
- * [完成] 你不确定问题大不大，想看最终结果再判断

- 现在就拽回来的情况

- * [错误] 方向错了，继续跑也是白费（比如理解错了任务目标）
- * [错误] 某一步一直失败，Agent 在无限重试（比如网站彻底挂了）
- * [错误] 你发现素材给错了（比如文件夹路径不对）
- * [错误] Agent 要做高风险操作，但没有先问你（比如要删除文件）

判断原则：早发现早中断，别等到最后才发现白干。

一个技巧：在

里加一句"每完成一个大步骤，向我汇报一次"。这样你能实时掌握进度，及时纠偏。

Brief

本章小结

- * Agent 执行任务的标准流程：理解 -> 计划 -> 执行 -> 验证 -> 汇报

- * 任务树展示每一步的状态，[完成] 成功 / [警告] 警告 / [错误] 失败 / □ 等待
- * 中断：方向不对或卡住时立刻停；纠偏：在当前基础上调整细节；回滚：撤销不可逆操作（但不是所有操作都能回滚）
- * 判断“继续跑”还是“拽回来”：方向对就让它跑，方向错或卡住就立刻中断

第 5 章 沙箱与工具调用：能力边界

5.1 沙箱是安全网，不是限制

很多人第一次听到“沙箱”，会觉得是“限制 Agent 能力”。其实恰恰相反，沙箱是为了让你敢放心用 Agent。

什么是沙箱

沙箱（Sandbox）是一个隔离的运行环境，Agent 在里面做的所有操作，都不会影响沙箱外的系统。

类比：沙箱就像“试衣间”，你在里面试衣服，外面的人看不到；试完不满意，脱下来就行，不会有任何影响。

WorkBuddy 的沙箱机制

WorkBuddy 的沙箱包含三层防护：

防护层	作用	举例
文件隔离	Agent 只能访问你授权的文件夹	你授权“桌面”，Agent 就碰不到“我的文档”
操作审计	所有文件操作都有日志，可追溯	Agent 删了什么、改了什么，都能查到
关键操作确认	高风险操作（删除、发送）需要你确认	Agent 要删文件前，会先问你

沙箱的价值

没有沙箱，你不敢让 Agent 做这些事：

- * [错误] 整理文件（万一删错了？）
- * [错误] 批量重命名（万一改错了？）
- * [错误] 发送邮件（万一发错人了？）

有了沙箱，你可以放心交付：

- * [完成] Agent 只能在授权范围内操作
- * [完成] 关键操作会先问你
- * [完成] 出错了可以回滚

沙箱不是限制，而是信任的基础。

5.2 本地文件 / 浏览器 / 外部 API 的边界

Agent 的“手脚”（工具）有三大类，每类都有明确的边界。

● 本地文件操作

能做什么：

- * [完成] 读取文件（TXT、Word、Excel、PDF、图片）
- * [完成] 写入文件（生成报告、表格、PPT）
- * [完成] 移动、重命名、复制文件
- * [完成] 创建文件夹、整理文件
- * [完成] 格式转换（PDF 转 Word、图片转文字）

不能做什么：

- * [错误] 访问未授权的文件夹（比如系统文件夹）
- * [错误] 修改正在使用的文件（比如你正在编辑的 Word）
- * [错误] 绕过操作系统的权限限制

边界在哪：你授权哪些文件夹，Agent 就只能访问哪些。默认授权：桌面、文档、下载。

● 浏览器操作

能做什么：

- * [完成] 搜索信息（Google、百度、必应）
- * [完成] 抓取网页内容（新闻、文章、数据）
- * [完成] 填写表单（比如批量提交问卷）
- * [完成] 截图保存
- * [完成] 下载文件

不能做什么：

- * [错误] 绕过登录（Agent 不会帮你输入密码）
- * [错误] 破解反爬机制（遇到验证码会失败）
- * [错误] 访问需要特殊权限的页面（比如公司内网）

边界在哪： Agent 只能访问公开网页。需要登录的网站，你得先登录，然后把 Cookie 给 Agent（高级用法，第 6 章会讲）。

● 外部 API 调用

能做什么：

- * [完成] 调用公开 API（天气、汇率、地图）
- * [完成] 调用企业内部 API（如果你配置了 MCP）
- * [完成] 读写数据库（如果你授权了）

不能做什么：

- * [错误] 调用未授权的 API
- * [错误] 绕过 API 的访问限制（比如超出调用次数）

边界在哪： 你配置了哪些 API，Agent 就能调用哪些。默认只有公开 API。

● 5.3 给敏感数据画围栏

Agent 很强大，但不是所有数据都适合交给它处理。

● 三类敏感数据

类型	举例	风险
机密信息	财务报表、战略规划、客户名单	泄露后影响公司利益
个人隐私	身份证号、银行卡号、家庭住址	泄露后影响个人安全
合规数据	医疗记录、法律文件、审计材料	处理不当可能违法

● 如何画围栏

方法 1：物理隔离

把敏感数据放在 Agent 访问不到的文件夹。

比如：

- * Agent 授权访问"桌面"和"文档"

* 敏感数据放在"D:\Confidential"（未授权）

方法 2：脱敏处理

如果必须让 Agent 处理敏感数据，先脱敏。

比如：

* 原始数据：张三，身份证 110101199001011234

* 脱敏后：张三，身份证 1101011990

方法 3：人工复核

敏感数据处理完，必须人工复核，不能直接用 Agent 的输出。

比如：

* Agent 生成财务报表 -> 你复核数字是否正确 -> 再提交给老板

● 一个原则

如果数据泄露后你会被开除，就别交给 Agent。

Agent 不是万能的，也不是绝对安全的。敏感数据的处理，永远需要人类把关。

● 5.4 不可逆操作的双重确认

有些操作一旦执行，就无法撤销。这类操作必须双重确认。

● 不可逆操作清单

操作	为什么不可逆	举例
删除文件	文件彻底删除（不在回收站）	批量删除重复文件
发送消息	消息已送达，无法撤回	发送邮件、发送企业微信
调用外部 API	操作已执行，无法撤销	扣款、发货、发布公告
修改数据库	数据已改，难以恢复	批量更新用户状态

● 双重确认机制

第一重：Agent 主动确认

在 Brief 里写明：

> "如果要删除文件或发送消息，必须先列出清单，等我确认后再执行"

Agent 会先展示：

```
1  【待删除文件清单】
2  1. duplicate_1.pdf（重复文件）
3  2. duplicate_2.pdf（重复文件）
4  3. old_report.docx（超过 1 年未修改）
5
6  共 3 个文件，总大小 15MB
7
8  【确认】是否删除以上文件？
```

你确认后，Agent 才会执行。

第二重：系统强制确认

WorkBuddy 对高风险操作有系统级确认。即使你在 Brief 里说"直接删"，系统也会弹窗让你确认。

● 一个建议

第一次用 Agent 做不可逆操作，先用测试数据试一遍。

比如：

- * 要批量删除文件？先在测试文件夹里试
- * 要批量发邮件？先发给自己试
- * 要调用 API？先用测试环境试

试成功了，再用真实数据。

本章小结

- * 沙箱是安全网，不是限制：文件隔离 + 操作审计 + 关键操作确认
- * Agent 的三大工具：本地文件（授权文件夹）、浏览器（公开网页）、外部 API（配置后可用）
- * 敏感数据要画围栏：物理隔离、脱敏处理、人工复核
- * 不可逆操作要双重确认：Agent 主动确认 + 系统强制确认，第一次用测试数据试

第 6 章 技能包与 MCP：让 Agent 长出手脚

6.1 20+ 预置技能包：怎么选、怎么组合

WorkBuddy 预置了 20+ 技能包，每个技能包是一组工具的集合。

技能包分类

类别	技能包	核心能力	适合场景
文档处理	Word 自动化	生成、编辑、格式化 Word	周报、合同、方案
表格数据	Excel 自动化	公式计算、数据透视、图表	财务对账、数据分析
信息获取	网页抓取	抓取新闻、文章、数据	竞品监控、行业调研
办公协作	邮件起草	根据要点生成邮件	客户沟通、内部通知

怎么选技能包

原则 1：按任务目标选

不要看"这个技能包好像很厉害"，而要看"我的任务需要什么"。

举例：

- * 任务：整理发票 -> 选"PDF 解析" + "Excel 自动化"
- * 任务：竞品周报 -> 选"网页抓取" + "Word 自动化"
- * 任务：客户邮件 -> 选"邮件起草"

原则 2：优先用预置，再考虑自定义

预置技能包已经覆盖 80% 的办公场景。别一上来就想"我要自己写一个"。

怎么组合技能包

单个技能包能力有限，组合使用才能发挥威力。

组合模式 1：串联

一个技能包的输出，是下一个技能包的输入。

举例：竞品周报

1 网页抓取（抓新闻） → 数据清洗（提取关键信息） → Word 自动化（生成报告）

组合模式 2：并联

多个技能包同时工作，最后汇总。

举例：多渠道调研

1

网页抓取 (36kr) ↓

2

搜索引擎 (Google) ↓ 数据清洗 → Excel 自动化

3

PDF 解析 (行业报告) ↓

组合模式 3：循环

重复执行同一组技能包。

举例：每周竞品监控

1 每周一 → 网页抓取 → 数据清洗 → Word 自动化 → 发送邮件 → 等待下周

6.2 MCP 的本质：给 Agent 装插件

MCP (Model Context Protocol) 是 Anthropic 推出的 Agent 扩展协议，让 Agent 能调用外部工具和服务。

MCP 是什么

类比：MCP 就像手机的"应用商店"，你可以给 Agent 安装各种"App"。

- * 预置技能包 = 系统自带 App (相机、计算器)
- * MCP 插件 = 第三方 App (微信、抖音)

MCP 能做什么

MCP 类型	举例	用途
数据源	数据库、CRM、ERP	让 Agent 读取企业数据
工具	翻译、OCR、语音转文字	扩展 Agent 的处理能力
服务	企业微信、钉钉、飞书	让 Agent 和企业系统打通
平台	NocoBase、Airtable	让 Agent 操作低代码平台

MCP 生态

截至 2026 年 5 月，MCP 生态已有 500+ 插件，覆盖：

- * 办公协作：企业微信、飞书、Slack
- * 数据分析：Tableau、Power BI、Metabase

- * 开发工具: GitHub、GitLab、Jira
- * 营销工具: HubSpot、Salesforce、Google Analytics

WorkBuddy 兼容 OpenClaw 生态, 可以直接使用 OpenClaw 的 MCP 插件。

6.3 接入企业系统的三种路径

把 Agent 和企业系统打通, 有三种路径, 成本和效果不同。

• 路径 1: 低成本 - 用现成 MCP (推荐)

适合场景: 企业用的是主流 SaaS (企业微信、飞书、钉钉、Salesforce)

怎么做:

1. 在 MCP 市场找到对应插件
2. 安装并授权
3. 在 Brief 里直接调用

举例: 接入企业微信

1. 安装"企业微信 MCP"插件
2. 用企业微信管理员账号授权
3. Brief 里写: "从企业微信拉取本周的客户咨询记录, 整理成 Excel"

成本: 10 分钟配置, 0 代码

• 路径 2: 中成本 - 用 API 桥接

适合场景: 企业系统有 API, 但没有现成 MCP

怎么做:

1. 找到企业系统的 API 文档
2. 用 MCP 开发工具 (比如 MiniClaw) 包装 API
3. 安装到 WorkBuddy

举例: 接入内部 CRM

1. CRM 有 REST API, 可以查询客户信息
2. 用 MiniClaw 写一个 MCP 插件, 封装 API 调用
3. WorkBuddy 就能直接"从 CRM 查询客户"

成本：1-2 天开发，需要懂 API

- 路径 3：高成本 - 定制开发

适合场景：企业系统没有 API，或者需要复杂的业务逻辑

怎么做：

1. 找开发团队定制 MCP 插件
2. 可能需要改造企业系统，暴露接口
3. 部署到 WorkBuddy

举例：接入老旧 ERP

1. ERP 是 10 年前的系统，没有 API
2. 开发团队给 ERP 加一个 API 层
3. 再开发 MCP 插件对接

成本：1-4 周开发，需要开发团队

- 选择建议

- * 80% 的场景用路径 1（现成 MCP）就够了
- * 15% 的场景需要路径 2（API 桥接）
- * 只有 5% 的场景需要路径 3（定制开发）

别一上来就想定制，先看看有没有现成的。

6.4 什么时候自己写、什么时候别碰

很多技术同学看到 MCP，第一反应是“我来写一个”。但大部分时候，这是浪费时间。

- 什么时候该自己写

[完成] 场景 1：企业特有流程

比如：公司有一套独特的审批流程，市面上没有现成工具。

[完成] 场景 2：数据敏感，不能用第三方

比如：财务数据不能传到外部 MCP 服务器，必须本地处理。

[完成] 场景 3：高频使用，值得投入

比如：每天要用 100 次，写一个 MCP 能省大量时间。

● 什么时候别碰

[错误] 场景 1：已经有现成 MCP

别重复造轮子。MCP 市场有 500+ 插件，90% 的需求都有现成方案。

[错误] 场景 2：低频使用，不值得投入

比如：一个月用一次，手动做 5 分钟，写 MCP 要 2 天。

[错误] 场景 3：需求不明确

别"先写了再说"。需求不明确的 MCP，写出来也没人用。

● 一个建议

先用 3 个月预置技能包，再考虑自己写 MCP。

用 3 个月，你会发现：

- * 哪些任务高频出现
- * 哪些任务预置技能包搞不定
- * 哪些任务值得投入开发

这时候再写 MCP，方向才是对的。

本章小结

- * WorkBuddy 预置 20+ 技能包，覆盖文档、表格、信息获取、办公协作
- * 技能包可以串联（A→B→C）、并联（A+B→C）、循环（每周重复）
- * MCP 是 Agent 的"应用商店"，可以安装第三方插件扩展能力
- * 接入企业系统三种路径：现成 MCP（10分钟）、API 桥接（1-2天）、定制开发（1-4周）
- * 80% 场景用现成 MCP，别急着自己写

第 7 章 workflow 设计：一次性 → 可复用 → 自动化

● 7.1 三种任务形态的识别

同样是"做周报"，有三种不同的形态，处理方式完全不同。

形态	特征	举例	处理方式
一次性任务	只做一次，不会重复	给新客户写一封介绍邮件	直接写 Brief，做完就完
可复用任务	会重复做，但不定期	每次有新员工入职，发欢迎邮件	沉淀为模板，需要时调用
自动化任务	定期重复，规则明确	每周一早上 9 点生成周报	配置触发器，自动执行

● 如何识别

问自己三个问题：

- 1. 这个任务会重复做吗？ -> 是 -> 可复用或自动化
- 2. 重复的频率固定吗？ -> 是 -> 自动化，否 -> 可复用
- 3. 任务的输入和输出每次都一样吗？ -> 是 -> 自动化，否 -> 可复用

● 7.2 把"每周都要做"固化为流程

可复用任务的核心是模板化。

● 模板化四步骤

第 1 步：做一次，记录过程

第一次做的时候，把 Brief、执行过程、遇到的问题都记下来。

第 2 步：提取变量

找出每次会变的部分，标记为变量。

举例：竞品周报

```
1  【变量】
2  - 日期范围: {{start_date}} 到 {{end_date}}
3  - 竞品列表: {{competitors}}
4
5  【固定】
6  - 数据源: 36kr、虎嗅
7  - 输出格式: Word, 使用 report_template.docx
8  - 保存位置: 桌面 reports 文件夹
```

第 3 步：写成模板 Brief

- 1 【任务】生成竞品动态周报
- 2 【日期范围】{{start_date}} 到 {{end_date}}
- 3 【竞品】{{competitors}}
- 4 【数据源】36kr、虎嗅
- 5 【输出】Word 文档，使用 report_template.docx，保存到桌面 reports 文件夹

第 4 步：测试和优化

用模板做 2-3 次，看看有没有遗漏或需要调整的地方。

● 模板库的组织

建议按职能分类：

- 1 我的 Agent 模板库/
- 2 | 市场/
- 3 | | 竞品周报.md
- 4 | | 行业调研.md
- 5 | | 活动复盘.md
- 6 | 销售/
- 7 | | 线索打分.md
- 8 | | 客户跟进.md
- 9 | 财务/
- 10 | | 月度对账.md
- 11 | | 发票整理.md
- 12 | 通用/
- 13 | | 文件整理.md
- 14 | | 邮件起草.md

7.3 多 Agent 协同：让专业的人做专业的事

复杂任务往往需要多个 Agent 协同完成。

● 协同模式 1：流水线

Agent A 做完 -> 传给 Agent B -> 传给 Agent C

举例：客户调研报告

- 1 Agent A (数据收集) : 抓取客户反馈、问卷结果
- 2 ↓
- 3 Agent B (数据分析) : 提取主题、统计分布
- 4 ↓
- 5 Agent C (报告生成) : 生成 PPT, 包含图表和结论

● 协同模式 2: 并行

多个 Agent 同时工作, 最后汇总。

举例: 多渠道竞品监控

- 1 Agent A: 监控 36kr
- 2 Agent B: 监控虎嗅
- 3 Agent C: 监控竞品官网
- 4 ↓
- 5 Agent D: 汇总所有信息, 生成周报

● 协同模式 3: 主从

一个主 Agent 调度多个子 Agent。

举例: 季度业务复盘

- 1 主 Agent: 制定复盘计划
- 2 |— 子 Agent 1: 拉取销售数据
- 3 |— 子 Agent 2: 拉取财务数据
- 4 |— 子 Agent 3: 拉取用户反馈
- 5 |— 主 Agent: 汇总分析, 生成报告

● WorkBuddy 的协同机制

WorkBuddy 支持"任务链"功能:

- 1 【任务链】客户调研报告
- 2 |— 任务 1: 数据收集 (Agent A)
- 3 |— 任务 2: 数据分析 (Agent B, 依赖任务 1)
- 4 |— 任务 3: 报告生成 (Agent C, 依赖任务 2)

你只需要定义任务链, WorkBuddy 会自动调度。

7.4 触发器、调度、异常处理

自动化任务需要三个机制：触发器（什么时候开始）、调度（怎么执行）、异常处理（出错了怎么办）。

● 触发器类型

类型	说明	举例
定时触发	固定时间执行	每周一早上 9 点生成周报
事件触发	某个事件发生时执行	收到新邮件时，自动分类
条件触发	满足某个条件时执行	库存低于 100 时，发送预警

● 调度策略

串行调度：一个任务做完再做下一个（适合有依赖关系的任务）

并行调度：多个任务同时做（适合独立任务）

优先级调度：重要任务先做（适合资源有限的情况）

● 异常处理

自动化任务最怕“半夜出错，没人管”。必须设计异常处理机制。

策略 1：重试

网络超时、临时故障 -> 自动重试 3 次

策略 2：降级

主数据源失败 -> 切换到备用数据源

策略 3：告警

重试 3 次仍失败 -> 发送告警通知（邮件/企业微信）

策略 4：跳过

某个子任务失败 -> 跳过并继续，最后汇总失败清单

● 自动化任务的监控

建议每周检查一次自动化任务的执行日志：

- * 成功率是多少
- * 哪些任务经常失败
- * 平均执行时间是否变长

发现问题及时调整。

本章小结

- * 三种任务形态：一次性（直接做）、可复用（模板化）、自动化（配置触发器）
- * 模板化四步骤：做一次记录 -> 提取变量 -> 写成模板 -> 测试优化
- * 多 Agent 协同三种模式：流水线（串行）、并行（同时）、主从（调度）
- * 自动化任务三要素：触发器（何时开始）、调度（如何执行）、异常处理（出错怎么办）

第 8 章 跨职能场景深度解剖

这一章是全书的核心。我们选了 5 个跨职能场景，每个场景都会完整展示：Brief 设计 -> 执行观察 -> 结果复盘 -> 固化复用。

8.1 竞品动态周报（市场）

● 场景背景

你是市场部的竞品分析师，每周一需要给老板提交一份竞品动态周报。老板关心三件事：

1. 竞品有什么新动作（融资、新品、市场活动）
2. 这些动作对我们有什么影响
3. 我们该如何应对

以前你要花 3 小时：

- * 1 小时：在 36kr、虎嗅、竞品官网上找新闻
- * 1 小时：整理成结构化信息
- * 1 小时：写分析和建议

现在用 Agent，目标是 30 分钟完成。

● Brief 设计

- 1 【背景】
- 2 我是市场部竞品分析师，每周需要提交竞品动态周报给老板。
- 3 我们是 SaaS 项目管理工具，主要竞品是 Asana、Monday、Notion。
- 4
- 5 【任务】
- 6 目标：生成本周（5月1-7日）竞品动态周报
- 7 约束：
- 8 - 只看 Asana、Monday、Notion 三家
- 9 - 每家至少 3 条动态，如果某家没有动态，标注"本周无重大动态"
- 10 - 动态类型：融资、新品发布、市场活动、高管变动、用户增长数据
- 11 - 按时间倒序排列（最新的在前）
- 12 素材：
- 13 - 数据源：36kr、虎嗅、TechCrunch、竞品官网新闻页
- 14 - 参考上周周报格式：desktop/reports/competitor_weekly_2026-04-30.docx
- 15 验收：
- 16 - 生成 Word 文档，包含：
- 17 - 竞品名称
- 18 - 动态标题
- 19 - 时间
- 20 - 来源链接
- 21 - 一句话总结（50 字以内）
- 22 - 对我们的影响（高/中/低）
- 23 - 建议应对措施（可选）
- 24
- 25 【交付】
- 26 - 文件名：competitor_weekly_2026-05-07.docx
- 27 - 保存位置：desktop/reports/
- 28 - 格式：Word，使用 report_template.docx 模板
- 29
- 30 【执行要求】
- 31 先列出详细计划，包括每一步要做什么、预计耗时、可能的风险，等我确认后再执行。

● 执行观察

Agent 列出的计划：

- 1 【计划】
- 2 1. 抓取数据源（预计 2 分钟）
- 3 - 36kr 搜索 "Asana OR Monday OR Notion", 时间范围 5月1-7日
- 4 - 虎嗅同样搜索
- 5 - TechCrunch 搜索
- 6 - 访问三家竞品官网的新闻页
- 7
- 8 2. 提取关键信息（预计 1 分钟）
- 9 - 筛选出符合 "动态类型" 的新闻
- 10 - 提取标题、时间、来源链接
- 11 - 生成一句话总结
- 12
- 13 3. 影响评估（预计 1 分钟）
- 14 - 根据动态类型判断影响程度：
- 15 - 融资/新品发布 → 高
- 16 - 市场活动 → 中
- 17 - 其他 → 低
- 18 - 生成建议应对措施（如果影响为 "高"）
- 19
- 20 4. 生成报告（预计 30 秒）
- 21 - 按 report_template.docx 格式生成
- 22 - 按时间倒序排列
- 23 - 保存到 desktop/reports/
- 24
- 25 【风险】
- 26 - 如果某个网站访问失败，会跳过并在报告里标注
- 27 - 如果某家竞品本周确实没有动态，会标注 "本周无重大动态"
- 28
- 29 【确认】以上计划是否 OK？

你确认后，Agent 开始执行。

任务树：

```
1  生成竞品动态周报
2  └─ 1. 抓取 36kr (找到 5 条相关新闻, 耗时 25s)
3  └─ 2. 抓取虎嗅 (找到 3 条相关新闻, 耗时 18s)
4  └─ 3. 抓取 TechCrunch (访问超时, 已跳过)
5  └─ 4. 抓取 Asana 官网 (找到 2 条新闻, 耗时 12s)
6  └─ 5. 抓取 Monday 官网 (找到 1 条新闻, 耗时 10s)
7  └─ 6. 抓取 Notion 官网 (本周无新闻)
8  └─ 7. 提取关键信息 (处理 11 条新闻, 耗时 15s)
9  └─ 8. 影响评估 (标注 2 条"高影响", 耗时 8s)
10 └─ 9. 生成报告 (保存到 desktop/reports/, 耗时 5s)
11
12  警告:
13 - TechCrunch 访问超时, 已跳过
14 - Notion 本周无新闻, 已标注
15
16  任务完成, 报告已生成: desktop/reports/competitor_weekly_2026-05-07.docx
```

● 结果复盘

做对了什么:

- * [完成] 自动抓取多个数据源, 省去手动搜索时间
- * [完成] 按模板生成结构化报告, 格式统一
- * [完成] 自动评估影响程度, 不用每条都人工判断

遇到的问题:

- * [警告] TechCrunch 访问超时: 可能是网络问题, 下次可以加"重试 3 次"
- * [警告] Notion 本周无新闻: 这是正常情况, Agent 正确标注了

可以优化的地方:

- * [提示] "建议应对措施"部分比较泛泛, 可以在 Brief 里加更多背景信息 (比如我们的产品路线图), 让 Agent 给出更有针对性的建议
- * [提示] 可以加一个"竞品对比表", 横向对比三家的动态数量和类型

时间对比:

- * 人工: 3 小时
- * Agent: 5 分钟 (含确认计划)
- * 节省: 2 小时 55 分钟

● 固化复用

把这个 Brief 保存为模板：

```
1 【模板名称】竞品动态周报
2 【变量】
3 - 日期范围: {{start_date}} 到 {{end_date}}
4 - 竞品列表: {{competitors}} (默认: Asana, Monday, Notion)
5 - 数据源: {{sources}} (默认: 36kr, 虎嗅, TechCrunch, 竞品官网)
6
7 【固定内容】
8 - 动态类型: 融资、新品发布、市场活动、高管变动、用户增长数据
9 - 输出格式: Word, 使用 report_template.docx
10 - 保存位置: desktop/reports/
11 - 影响评估: 融资/新品 → 高, 市场活动 → 中, 其他 → 低
12
13 【调用方式】
14 "用竞品周报模板, 日期范围 {{start_date}} 到 {{end_date}}"
```

下次只需要说: > "用竞品周报模板, 日期范围 5月8日到5月14日"

Agent 自动填充变量, 执行任务。

● 进阶: 自动化

如果每周一都要做, 可以配置自动化:

```
1 【触发器】每周一早上 8:00
2 【任务】用竞品周报模板, 日期范围为"上周一到上周日"
3 【完成后】发送邮件给老板, 附件为生成的报告
```

这样你周一上班, 报告已经在老板邮箱里了。

● 8.2 销售线索打分与跟进（销售）

● 场景背景

你是销售团队的 SDR（销售开发代表），每天会收到 50-100 条销售线索（来自官网表单、活动报名、广告投放）。

你的任务是：

1. 快速筛选出高质量线索
2. 给每条线索打分（A/B/C 级）
3. 生成跟进话术

以前你要花 2 小时：

- * 1 小时：逐条看线索信息，判断质量
- * 30 分钟：打分和分类
- * 30 分钟：写跟进话术

现在用 Agent，目标是 20 分钟完成。

● Brief 设计

- 1 【背景】
- 2 我是 SDR，负责筛选和跟进销售线索。
- 3 我们的产品是企业级 SaaS 项目管理工具，目标客户是 50 人以上的科技公司。
- 4
- 5 【任务】
- 6 目标：筛选并打分今天的销售线索
- 7 约束：
- 8 - 打分标准：
- 9 - A 级（高优先级）：公司规模 50人+、科技行业、有明确需求、决策人
- 10 - B 级（中优先级）：满足 2-3 个条件
- 11 - C 级（低优先级）：满足 0-1 个条件
- 12 - 排除：个人用户、学生、明确表示“只是看看”
- 13 素材：
- 14 - 线索数据：desktop/leads/leads_2026-05-08.csv
- 15 - 字段：姓名、公司、职位、公司规模、行业、需求描述、来源
- 16 验收：
- 17 - 生成 Excel 表格，包含：
- 18 - 原始线索信息
- 19 - 评分（A/B/C）
- 20 - 评分理由（一句话）
- 21 - 跟进话术（50 字以内）
- 22 - 建议跟进时间（今天/本周/下周）
- 23
- 24 【交付】
- 25 - 文件名：leads_scored_2026-05-08.xlsx
- 26 - 保存位置：desktop/leads/
- 27 - 格式：Excel，按评分排序（A → B → C）
- 28
- 29 【执行要求】
- 30 先列出详细计划，等我确认后再执行。

● 执行观察

Agent 列出的计划：


```
1  【计划】
2  1. 读取线索数据（预计 5 秒）
3      - 从 leads_2026-05-08.csv 读取所有线索
4      - 检查字段完整性
5
6  2. 筛选和打分（预计 30 秒）
7      - 逐条分析线索信息
8      - 根据打分标准评分
9      - 排除不符合条件的线索
10
11 3. 生成跟进话术（预计 1 分钟）
12     - 根据需求描述和公司背景，生成个性化话术
13     - A 级：强调产品价值和 ROI
14     - B 级：询问具体需求
15     - C 级：提供免费试用
16
17 4. 生成 Excel（预计 10 秒）
18     - 按评分排序
19     - 保存到 desktop/leads/
20
21 【风险】
22 - 如果某条线索信息不完整（比如缺少公司规模），会标注"信息不足，建议人工复核"
23
24 【确认】以上计划是否 OK？
```

你确认后，Agent 开始执行。

任务树：

```
1  销售线索打分
2  └─ 1. 读取线索数据（共 87 条线索，耗时 3s）
3  └─ 2. 筛选和打分（耗时 25s）
4  │   └─ A 级：12 条
5  │   └─ B 级：35 条
6  │   └─ C 级：28 条
7  │   └─ 排除：12 条（个人用户、学生）
8  └─ 3. 生成跟进话术（耗时 45s）
9  └─ 4. 生成 Excel（保存到 desktop/leads/, 耗时 5s）
10
11 任务完成，共筛选出 75 条有效线索（12 条 A 级，35 条 B 级，28 条 C 级）
```

● 结果复盘

做对了什么：

* [完成] 自动筛选掉无效线索（个人用户、学生），省去人工判断时间

- * [完成] 打分标准清晰，结果稳定
- * [完成] 跟进话术个性化，不是千篇一律的模板

遇到的问题：

- * [警告] 有 3 条线索信息不完整，Agent 标注了"信息不足"，需要人工复核

可以优化的地方：

- * [提示] 跟进话术可以更个性化，比如提到对方公司最近的新闻或融资
- * [提示] 可以加一个"相似客户案例"字段，帮助销售快速找到参考

时间对比：

- * 人工：2 小时
- * Agent：3 分钟（含确认计划）
- * 节省：1 小时 57 分钟

● 固化复用

保存为模板：

```
1  【模板名称】销售线索打分
2  【变量】
3  - 线索文件: {{leads_file}}
4  - 打分标准: {{scoring_criteria}}（可自定义）
5
6  【固定内容】
7  - 输出格式: Excel，按评分排序
8  - 保存位置: desktop/leads/
9  - 跟进话术: 根据评分生成个性化话术
10
11 【调用方式】
12 "用销售线索打分模板，线索文件 {{leads_file}}"
```

● 进阶：CRM 集成

如果公司用 Salesforce 或 HubSpot，可以通过 MCP 直接对接：

- 1 【任务】从 Salesforce 拉取今天的新线索，打分后自动更新回 Salesforce
- 2 【触发器】每天下午 5:00
- 3 【完成后】发送日报给销售主管，汇总今天的 A 级线索

● 8.3 月度对账与异常标注（财务）

● 场景背景

你是财务部的对账专员，每月初需要核对上个月的收支流水。

你的任务是：

1. 核对银行流水和财务系统记录是否一致
2. 标注异常项（金额不符、缺失记录、重复记录）
3. 生成对账报告

以前你要花 4 小时：

- * 2 小时：逐条核对流水
- * 1 小时：标注异常
- * 1 小时：生成报告

现在用 Agent，目标是 30 分钟完成。

● Brief 设计

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15
- 16
- 17
- 18
- 19
- 20
- 21
- 22
- 23
- 24
- 25
- 26
- 27
- 28
- 29
- 30
- 31
- 32
- 【背景】
 - 我是财务对账专员，每月初需要核对上个月的银行流水和财务系统记录。
 - 【任务】
 - 目标：核对 2026 年 4 月的银行流水和财务系统记录
 - 约束：
 - 核对规则：
 - 金额差异 > 0.01 元 → 标注"金额不符"
 - 银行流水有、财务系统没有 → 标注"缺失记录"
 - 财务系统有、银行流水没有 → 标注"待确认"
 - 同一笔交易出现多次 → 标注"疑似重复"
 - 排除：内部转账、测试交易
 - 素材：
 - 银行流水：desktop/finance/bank_statement_2026-04.csv
 - 财务系统记录：desktop/finance/erp_records_2026-04.csv
 - 匹配字段：交易时间、金额、对方账户
 - 验收：
 - 生成 Excel 表格，包含：
 - 一致记录（绿色标注）
 - 异常记录（红色标注）
 - 异常类型
 - 差异金额（如果有）
 - 建议处理方式
 - 【交付】
 - 文件名：reconciliation_2026-04.xlsx
 - 保存位置：desktop/finance/
 - 格式：Excel，分两个 Sheet：一致记录、异常记录
 - 【执行要求】
 - 先列出详细计划，等我确认后再执行。
 - 如果发现异常记录超过 10 条，先暂停并向我汇报。

● 执行观察

Agent 列出的计划：

```
1  【计划】
2  1. 读取数据（预计 10 秒）
3      - 读取银行流水（bank_statement_2026-04.csv）
4      - 读取财务系统记录（erp_records_2026-04.csv）
5      - 检查字段格式
6
7  2. 数据清洗（预计 20 秒）
8      - 统一日期格式
9      - 统一金额格式（保留 2 位小数）
10     - 排除内部转账和测试交易
11
12 3. 匹配核对（预计 1 分钟）
13     - 按"交易时间 + 金额 + 对方账户"匹配
14     - 标注一致记录
15     - 标注异常记录
16
17 4. 异常分析（预计 30 秒）
18     - 分类异常类型
19     - 计算差异金额
20     - 生成建议处理方式
21
22 5. 生成报告（预计 10 秒）
23     - 分两个 Sheet：一致记录、异常记录
24     - 用颜色标注（绿色/红色）
25     - 保存到 desktop/finance/
26
27 【风险】
28 - 如果异常记录超过 10 条，会先暂停并向你汇报
29 - 如果某条记录的字段缺失，会标注"数据不完整"
30
31 【确认】以上计划是否 OK？
```

你确认后，Agent 开始执行。

任务树：

```
1  月度对账
2  └─ 1. 读取数据（银行流水 523 条，财务系统 518 条，耗时 8s）
3  └─ 2. 数据清洗（排除内部转账 12 条，耗时 15s）
4  └─ 3. 匹配核对（耗时 45s）
5  │ └─ 一致记录：505 条
6  │ └─ 异常记录：6 条
7  └─ 4. 发现 6 条异常，是否继续？
8  │ （你确认"继续"）
9  └─ 5. 异常分析（耗时 20s）
10 │ └─ 金额不符：2 条（差异 0.05 元、0.12 元）
11 │ └─ 缺失记录：3 条（银行有、系统没有）
12 │ └─ 待确认：1 条（系统有、银行没有）
13 └─ 6. 生成报告（保存到 desktop/finance/，耗时 8s）
14
15 任务完成，共核对 511 条记录，发现 6 条异常
```

● 结果复盘

做对了什么：

- * [完成] 自动匹配 500+ 条记录，省去逐条核对的时间
- * [完成] 异常分类清晰，建议处理方式明确
- * [完成] 发现异常超过阈值时主动暂停，避免批量错误

遇到的问题：

- * [警告] 有 2 条"金额不符"的差异很小（0.05 元、0.12 元），可能是四舍五入导致的，需要人工判断是否需要调整

可以优化的地方：

- * [提示] 可以加一个"容差范围"，比如差异 < 0.1 元自动标注为"可忽略"
- * [提示] 可以生成"异常趋势图"，看看每月异常数量是否在增加

时间对比：

- * 人工：4 小时
- * Agent：3 分钟（含确认计划和异常确认）
- * 节省：3 小时 57 分钟

● 固化复用

保存为模板：

```
1  【模板名称】月度对账
2  【变量】
3  - 月份: {{month}}
4  - 银行流水文件: {{bank_file}}
5  - 财务系统文件: {{erp_file}}
6  - 异常阈值: {{threshold}} (默认 10 条)
7
8  【固定内容】
9  - 核对规则: 金额差异 > 0.01 元、缺失记录、重复记录
10 - 输出格式: Excel, 分两个 Sheet
11 - 保存位置: desktop/finance/
12
13 【调用方式】
14 "用月度对账模板, 月份 {{month}}"
```

● 进阶: 自动化 + 告警

```
1  【触发器】每月 1 号早上 9:00
2  【任务】用月度对账模板, 月份为"上个月"
3  【完成后】
4  - 如果异常记录 = 0: 发送邮件"上月对账无异常"
5  - 如果异常记录 > 0: 发送邮件 + 附件, 标题"上月对账发现 X 条异常, 请处理"
```

● 8.4 用户反馈主题洞察 (产品)

● 场景背景

你是产品经理, 每周需要整理用户反馈, 提取核心诉求。

你的任务是:

1. 从 200+ 条用户反馈中提取主题
2. 统计每个主题的提及次数
3. 标注优先级 (高/中/低)
4. 生成产品优化建议

以前你要花 3 小时:

- * 2 小时: 逐条看反馈, 手动归类
- * 1 小时: 统计和写建议

现在用 Agent, 目标是 20 分钟完成。

● Brief 设计

```

1  【背景】
2  我是产品经理，负责整理用户反馈并提取产品优化方向。
3  我们的产品是项目管理工具，用户主要是中小企业团队。
4
5  【任务】
6  目标：从本周用户反馈中提取主题并生成洞察报告
7  约束：
8  - 主题分类：功能需求、性能问题、UI/UX、Bug、其他
9  - 优先级判断：
10 - 高：提及次数 > 20 次，或影响核心功能
11 - 中：提及次数 10-20 次
12 - 低：提及次数 < 10 次
13 - 排除：无效反馈（辱骂、灌水、广告）
14 素材：
15 - 用户反馈：desktop/feedback/feedback_2026-W19.csv
16 - 字段：用户 ID、反馈内容、提交时间、来源（App/网页/客服）
17 验收：
18 - 生成 Excel 表格，包含：
19 - 主题名称
20 - 主题分类
21 - 提及次数
22 - 优先级
23 - 典型反馈（3 条）
24 - 产品优化建议
25
26 【交付】
27 - 文件名：feedback_insights_2026-W19.xlsx
28 - 保存位置：desktop/feedback/
29 - 格式：Excel，按优先级排序（高 → 中 → 低）
30
31 【执行要求】
32 先列出详细计划，等我确认后再执行。

```

● 执行观察

Agent 列出的计划:


```
1  【计划】
2  1. 读取反馈数据（预计 5 秒）
3      - 从 feedback_2026-W19.csv 读取所有反馈
4      - 排除无效反馈
5
6  2. 主题提取（预计 2 分钟）
7      - 用 NLP 分析反馈内容
8      - 提取关键词和主题
9      - 归类到 5 个分类
10
11 3. 统计和优先级标注（预计 30 秒）
12     - 统计每个主题的提及次数
13     - 根据次数和影响判断优先级
14
15 4. 生成建议（预计 1 分钟）
16     - 针对高优先级主题，生成产品优化建议
17     - 引用典型反馈作为依据
18
19 5. 生成报告（预计 10 秒）
20     - 按优先级排序
21     - 保存到 desktop/feedback/
22
23 【风险】
24 - 如果反馈内容太短或太模糊，可能无法准确提取主题，会标注"需人工复核"
25
26 【确认】以上计划是否 OK？
```

你确认后，Agent 开始执行。

任务树：

```
1  用户反馈主题洞察
2  └─ 1. 读取反馈数据（共 237 条，排除无效 15 条，耗时 4s）
3  └─ 2. 主题提取（提取出 12 个主题，耗时 1m 45s）
4  └─ 3. 统计和优先级标注（耗时 25s）
5      └─ 高优先级：3 个主题
6      └─ 中优先级：5 个主题
7      └─ 低优先级：4 个主题
8  └─ 4. 生成建议（针对 3 个高优先级主题，耗时 50s）
9  └─ 5. 生成报告（保存到 desktop/feedback/，耗时 6s）
10
11 任务完成，共提取 12 个主题，3 个高优先级需重点关注
```

● 结果复盘

做对了什么：

- * [完成] 自动从 200+ 条反馈中提取主题，省去手动归类时间
- * [完成] 优先级判断合理，高优先级主题确实是用户最关心的
- * [完成] 产品优化建议有依据（引用了典型反馈）

遇到的问题：

- * [警告] 有 2 个主题的命名比较模糊（"其他功能需求"），需要人工细化

可以优化的地方：

- * [提示] 可以加"情感分析"，标注用户是"抱怨"还是"建议"
- * [提示] 可以生成"主题趋势图"，对比上周的主题变化

时间对比：

- * 人工：3 小时
- * Agent：4 分钟（含确认计划）
- * 节省：2 小时 56 分钟

● 固化复用

保存为模板：

```
1  【模板名称】用户反馈主题洞察
2  【变量】
3  - 反馈文件：{{feedback_file}}
4  - 优先级阈值：{{threshold}}（默认：高 > 20 次，中 10-20 次）
5
6  【固定内容】
7  - 主题分类：功能需求、性能问题、UI/UX、Bug、其他
8  - 输出格式：Excel，按优先级排序
9  - 保存位置：desktop/feedback/
10
11 【调用方式】
12 "用用户反馈主题洞察模板，反馈文件 {{feedback_file}}"
```

● 进阶：实时监控

```
1  【触发器】每天下午 6:00
2  【任务】用用户反馈主题洞察模板，反馈文件为"今天的反馈"
3  【完成后】
4  - 如果出现新的高优先级主题：立即发送告警邮件给产品团队
5  - 否则：汇总到周报
```

● 8.5 技术选型调研（研发）

● 场景背景

你是技术负责人，团队要引入一个新的技术栈（比如消息队列），需要做技术选型。

你的任务是：

1. 调研 3-5 个候选方案
2. 对比关键指标（性能、成本、社区活跃度、学习曲线）
3. 给出推荐方案和理由

以前你要花 1 天：

- * 4 小时：搜索资料、看文档、看对比文章
- * 2 小时：整理对比表
- * 2 小时：写调研报告

现在用 Agent，目标是 1 小时完成。

● Brief 设计

```

1  【背景】
2  我是技术负责人，团队需要引入消息队列，候选方案是 Kafka、RabbitMQ、RocketMQ、Pulsar。
3  我们的场景：日均消息量 1000 万条，需要高可用、低延迟，团队对 Java 比较熟悉。
4
5  【任务】
6  目标：对比 4 个消息队列方案，生成技术选型报告
7  约束：
8  - 对比维度：
9    - 性能（吞吐量、延迟）
10   - 成本（开源/商业、运维成本）
11   - 社区活跃度（GitHub Star、最近更新）
12   - 学习曲线（文档质量、上手难度）
13   - 适用场景
14 - 每个维度给出具体数据或评分（1-5 分）
15 素材：
16 - 数据源：官方文档、GitHub、技术博客（InfoQ、掘金、Medium）
17 - 参考文章：搜索"Kafka vs RabbitMQ vs RocketMQ vs Pulsar 2026"
18 验收：
19 - 生成 Word 文档，包含：
20   - 对比表（4 个方案 × 5 个维度）
21   - 每个方案的优缺点
22   - 推荐方案（排序：第一推荐、第二推荐）
23   - 推荐理由（结合我们的场景）
24
25  【交付】
26 - 文件名：mq_selection_report_2026-05-08.docx
27 - 保存位置：desktop/research/
28 - 格式：Word，包含对比表和文字说明
29
30  【执行要求】
31 先列出详细计划，等我确认后再执行。
32 如果某个方案的数据找不到，标注"数据缺失"并说明原因。

```

● 执行观察

Agent 列出的计划:

```

1  【计划】
2  1. 搜索对比文章（预计 2 分钟）
3      - 搜索 "Kafka vs RabbitMQ vs RocketMQ vs Pulsar 2026"
4      - 找 3-5 篇权威文章
5
6  2. 抓取官方数据（预计 3 分钟）
7      - 访问 4 个方案的官网和 GitHub
8      - 提取性能数据、Star 数、最近更新时间
9
10 3. 整理对比表（预计 2 分钟）
11     - 按 5 个维度整理数据
12     - 给出评分（1-5 分）
13
14 4. 生成推荐（预计 2 分钟）
15     - 结合你的场景（日均 1000 万条、高可用、低延迟、Java 团队）
16     - 给出第一推荐和第二推荐
17     - 说明理由
18
19 5. 生成报告（预计 30 秒）
20     - 生成 Word 文档
21     - 保存到 desktop/research/
22
23 【风险】
24 - 如果某个方案的性能数据找不到，会标注 "数据缺失"
25 - 如果对比文章的数据有冲突，会标注 "数据存在争议"
26
27 【确认】以上计划是否 OK？

```

你确认后，Agent 开始执行。

任务树：

```

1  技术选型调研
2  └─ 1. 搜索对比文章（找到 4 篇权威文章，耗时 1m 50s）
3  └─ 2. 抓取官方数据（耗时 2m 40s）
4  │   └─ Kafka: GitHub 28k Star, 最近更新 2 天前
5  │   └─ RabbitMQ: GitHub 12k Star, 最近更新 5 天前
6  │   └─ RocketMQ: GitHub 21k Star, 最近更新 1 周前
7  │   └─ Pulsar: GitHub 14k Star, 最近更新 3 天前
8  └─ 3. 整理对比表（耗时 1m 30s）
9  └─ 4. 生成推荐（耗时 1m 45s）
10 │   └─ 第一推荐: Kafka
11 │   └─ 第二推荐: RocketMQ
12 └─ 5. 生成报告（保存到 desktop/research/, 耗时 20s）
13
14 任务完成，推荐 Kafka 作为首选方案

```

● 结果复盘

做对了什么：

- * [完成] 自动搜索和整理多个数据源，省去手动查找时间
- * [完成] 对比表数据完整，评分有依据
- * [完成] 推荐方案结合了实际场景，不是泛泛而谈

遇到的问题：

- * [警告] Pulsar 的性能数据在不同文章里有冲突，Agent 标注了"数据存在争议"

可以优化的地方：

- * [提示] 可以加"迁移成本"维度，评估从现有方案迁移到新方案的难度
- * [提示] 可以加"案例参考"，列出使用该方案的知名公司

时间对比：

- * 人工：1 天（8 小时）
- * Agent：10 分钟（含确认计划）
- * 节省：7 小时 50 分钟

● 固化复用

保存为模板：

```
1  【模板名称】技术选型调研
2  【变量】
3  - 技术类型: {{tech_type}}（如：消息队列、数据库、缓存）
4  - 候选方案: {{candidates}}
5  - 对比维度: {{dimensions}}（可自定义）
6  - 使用场景: {{scenario}}
7
8  【固定内容】
9  - 数据源：官方文档、GitHub、技术博客
10 - 输出格式：Word，包含对比表和推荐
11 - 保存位置：desktop/research/
12
13 【调用方式】
14 "用技术选型调研模板，技术类型 {{tech_type}}，候选方案 {{candidates}}"
```

● 进阶：定期更新

- 1 【触发器】每季度第一天
- 2 【任务】用技术选型调研模板，更新“消息队列”的对比数据
- 3 【完成后】对比上季度的数据，如果有重大变化（如新版本发布、性能提升），发送通知

第 8 章小结

我们通过 5 个跨职能场景，展示了 Agent 在不同领域的应用：

场景	职能	核心价值	时间节省
竞品动态周报	市场	自动抓取 + 结构化输出	2h 55min
销售线索打分	销售	自动筛选 + 个性化话术	1h 57min
月度对账	财务	自动匹配 + 异常标注	3h 57min
用户反馈洞察	产品	主题提取 + 优先级判断	2h 56min
技术选型调研	研发	多源整合 + 对比分析	7h 50min

共同特点：

- * 都是重复性高、规则明确、信息密集的任务
- * 都可以模板化，下次直接复用
- * 都可以自动化，定期执行

下一步：

- * 把这 5 个场景的模板保存到你的模板库
- * 根据你的实际工作，调整变量和约束
- * 尝试组合多个场景，形成更复杂的工作流

第 9 章 协作模式：从个人到组织

9.1 个人：建立自己的 Brief 库和技能库

Agent 用得好不好，取决于你有没有积累。

- Brief 库：你的“指令资产”

每次写的 Brief，都是一次投入。如果不保存，下次还得重新写。

建议的 Brief 库结构：

```
1 我的 Brief 库/  
2  └─ 日常/  
3    │ └─ 周报生成.md  
4    │ └─ 邮件起草.md  
5    └─ 文件整理.md  
6  └─ 市场/  
7    │ └─ 竞品周报.md  
8    │ └─ 行业调研.md  
9    └─ 活动复盘.md  
10 └─ 数据/  
11   │ └─ 月度对账.md  
12   │ └─ 数据清洗.md  
13   └─ 报表生成.md  
14 └─ 调研/  
15   │ └─ 技术选型.md  
16   │ └─ 竞品分析.md  
17   └─ 用户调研.md
```

每个 Brief 文件包含：

- * 模板正文（带变量标记）
- * 使用说明（什么时候用、怎么填变量）
- * 历史记录（上次用的时间、效果如何、有什么调整）

● 技能库：你常用的工具组合

记录你常用的技能包组合，下次直接调用。

```
1  【我的技能组合】  
2  
3  1. 竞品监控套件  
4    - 网页抓取 + 数据清洗 + Word 自动化  
5    - 适用：每周竞品周报  
6  
7  2. 数据分析套件  
8    - Excel 自动化 + 数据清洗 + 图表生成  
9    - 适用：月度数据报告  
10  
11 3. 内容生产套件  
12   - 搜索引擎 + 网页抓取 + Word 自动化  
13   - 适用：行业调研、技术选型
```


● 个人效率的三个阶段

阶段	特征	时间投入	产出
新手期	每次都写新 Brief	高（每次 10-15 分钟）	低
积累期	开始复用模板	中（每次 2-3 分钟）	中
成熟期	一键调用 + 自动化	低（每次 10 秒）	高

目标是尽快从新手期进入成熟期。关键动作：每次做完任务，花 2 分钟把 Brief 保存为模板。

● 9.2 团队：共享流程和技能包的治理

当团队里多个人都在用 Agent，就需要“治理”——统一标准、共享资源、避免重复劳动。

● 共享 Brief 库

问题：每个人都在写自己的 Brief，质量参差不齐，重复劳动。

解决方案：建立团队共享 Brief 库。

```
1 团队 Brief 库/
2  |─ 市场部/
3  |   |─ [标准] 竞品周报.md（经过验证的标准模板）
4  |   |─ [标准] 行业调研.md
5  |   |─ [实验] 社媒监控.md（还在测试中）
6  |─ 销售部/
7  |   |─ [标准] 线索打分.md
8  |   |─ [标准] 客户跟进.md
9  |─ 通用/
10 |   |─ [标准] 周报生成.md
11 |   |─ [标准] 会议纪要.md
```

治理规则：

- * [标准]：经过 3 次以上验证，质量稳定，全团队可用
- * [实验]：还在测试中，可能有问题，使用需谨慎
- * 版本管理：每次修改记录变更原因和日期
- * 负责人：每个模板有一个负责人，负责维护和更新

● 共享技能包

问题：某个同事开发了一个好用的 MCP 插件，但其他人不知道。

解决方案：建立团队技能包目录。

```
1 团队技能包目录/  
2  | 官方预置 (20+ 个)  
3  | 团队自研 (5 个)  
4  |   | CRM 数据拉取 (对接 Salesforce)  
5  |   | 内部审批流 (对接 OA 系统)  
6  |   | 企业微信通知 (对接企微 API)  
7  |   | 数据仓库查询 (对接 BI 系统)  
8  |   | 合同模板填充 (对接法务系统)  
9  | 第三方 MCP (10 个)  
10 |   | 飞书集成  
11 |   | Notion 同步  
12 |   | ...
```

● 团队协作的三个层次

层次	内容	适合团队规模
共享模板	统一 Brief 模板，减少重复劳动	3-10 人
共享技能包	统一工具集，降低接入成本	10-30 人
共享工作流	统一流程，跨角色协同	30+ 人

9.3 组织：权限、审计、合规

当 Agent 在组织级别使用，就涉及安全和合规问题。

● 权限管理

原则：最小权限。

每个人只能访问自己需要的数据和工具。

角色	文件权限	工具权限	API 权限
普通员工	个人文件夹	预置技能包	无
部门主管	部门文件夹	预置 + 部门自研	部门 API
IT 管理员	全部	全部	全部

● 审计日志

所有 Agent 操作都应该有日志，包括：

- * 谁在什么时间执行了什么任务
- * 访问了哪些文件
- * 调用了哪些 API
- * 输出了什么结果

审计日志的用途：

- * 安全事件追溯
- * 合规审查
- * 使用量统计（成本分摊）

● 合规要求

不同行业有不同的合规要求：

行业	合规要求	Agent 使用限制
金融	数据不出境、操作可追溯	必须本地部署、全量审计
医疗	患者隐私保护	不能处理患者个人信息
法律	律师-客户特权	不能将案件信息传到外部
政府	等保要求	必须通过安全认证

WorkBuddy 的合规优势：本地部署，数据不出设备，天然满足"数据不出境"要求。

● 组织级推广的三步走

第 1 步：试点（1-2 个月）

- * 选 1-2 个部门试点
- * 验证效果和安全性
- * 收集反馈

第 2 步：推广（2-3 个月）

- * 制定使用规范
- * 培训全员
- * 建立共享资源库

第 3 步：治理（持续）

- * 监控使用情况

- * 定期审计
- * 持续优化

本章小结

- * 个人层面：建立 Brief 库和技能库，从"每次重写"进化到"一键调用"
- * 团队层面：共享模板和技能包，统一标准，减少重复劳动
- * 组织层面：权限管理（最小权限）、审计日志（全量记录）、合规要求（按行业）
- * 推广路径：试点 -> 推广 -> 治理

第 10 章 判断力：什么时候不该用 Agent

10.1 Agent 幻觉比 ChatBot 更危险

ChatBot 的幻觉（编造信息）最多让你看到一个错误答案。Agent 的幻觉会直接执行错误操作。

● 幻觉的三种表现

类型	ChatBot 的后果	Agent 的后果
编造事实	你看到错误信息	Agent 基于错误信息做决策
理解偏差	回答不相关	执行错误操作
过度自信	语气肯定但内容错	不确认就直接执行

● 真实案例

案例 1：编造数据源

你让 Agent 抓取某个网站的数据，Agent 声称"已成功抓取 50 条数据"，但实际上网站已经改版，Agent 抓到的是旧页面的缓存数据。

案例 2：理解偏差导致误操作

你说"把重复文件删掉"，Agent 把"文件名相似"的文件也删了（比如 report_v1.docx 和 report_v2.docx），但它们其实是不同版本。

案例 3：过度自信

你让 Agent 发送邮件，Agent 没有确认收件人就直接发了，结果发错了人。

● 防范措施

- 1. 强制规划：让 Agent 先列计划，你确认后再执行
- 2. 关键操作确认：删除、发送、修改等操作必须确认
- 3. 结果验收：Agent 做完后，你必须检查结果
- 4. 测试先行：第一次做的任务，先用测试数据试

10.2 验收 vs 信任：永远的人机边界

Agent 再强大，也不能完全信任。你必须保持"验收"的习惯。

● 信任阶梯

信任等级	适用场景	验收方式
零信任	第一次做的任务	逐步确认，每一步都检查
有限信任	做过 3-5 次的任务	只检查最终结果
高信任	做过 10+ 次的任务	抽查（每 5 次检查 1 次）
完全信任	[错误] 不存在	永远不要完全信任

为什么不能完全信任？

- * 数据源可能变化（网站改版、API 升级）
- * 任务条件可能变化（新增竞品、规则调整）
- * Agent 模型可能更新（行为可能变化）

● 验收的最小成本

验收不等于"重新做一遍"。高效验收的方法：

方法 1：抽样检查

不看全部结果，只看 3-5 条关键数据。

方法 2：对比基准

和上次的结果对比，看有没有异常偏差。

方法 3：交叉验证

用另一个数据源验证 Agent 的结论。

方法 4：设置红线

定义"不可接受"的结果，Agent 触发红线时自动告警。

10.3 成本意识：token、时间、决策质量三本账

用 Agent 不是免费的，要算清楚三本账。

● 第一本账：Token 成本

Agent 每次执行任务都消耗 token（计算资源）。

任务类型	大约 token 消耗	大约成本
简单文件操作	1,000-5,000	□0.01-0.05
数据分析	5,000-20,000	□0.05-0.2
网页抓取 + 报告	20,000-100,000	□0.2-1.0
复杂调研	100,000-500,000	□1.0-5.0

省 token 的技巧：

- * Brief 写得越精确，Agent 试错越少，token 消耗越低
- * 用模板比每次重写省 30-50% token
- * 把大任务拆成小任务，失败时只重做失败的部分

● 第二本账：时间成本

Agent 节省的是"执行时间"，但你还需要投入"管理时间"。

1 总时间 = 写 Brief 时间 + Agent 执行时间 + 验收时间

如果"写 Brief + 验收"的时间 > 自己做的时间，那就不值得用 Agent。

什么时候 Agent 的时间 ROI 最高：

- * 任务执行时间长 (> 30 分钟)
- * Brief 可以复用（模板化）
- * 验收成本低（结果容易判断对错）

● 第三本账：决策质量

有些任务，Agent 的输出质量不如人类。

任务类型	Agent 质量	人类质量	谁更好
数据整理	95%	90%（人会犯错）	Agent
信息搜索	85%	95%（人更会判断）	人类
创意写作	60%	90%	人类
战略决策	40%	85%	人类

结论：

- * 数据密集型任务 -> Agent 更好（不会犯低级错误）
- * 判断密集型任务 -> 人类更好（需要经验和直觉）
- * 创意密集型任务 -> 人类更好（Agent 输出平庸）

10.4 6 类任务真的不划算

以下 6 类任务，用 Agent 的 ROI 很低，建议直接自己做。

● 1. 一次性的简单任务

举例：改一个文件名、发一封短邮件、查一个数据

为什么不划算：写 Brief 的时间 > 自己做的时间。

判断标准：如果自己做 < 2 分钟，别用 Agent。

● 2. 高度创意的任务

举例：写品牌 slogan、设计产品名称、构思营销创意

为什么不划算：Agent 输出平庸，你还得大改，不如自己想。

判断标准：如果任务的核心价值是“独特性”，别用 Agent。

● 3. 需要频繁人类决策的任务

举例：和客户谈判、面试候选人、处理投诉

为什么不划算：每一步都要你确认，来回沟通比自己做更慢。

判断标准：如果任务中 > 50% 的步骤需要你决策，别用 Agent。

● 4. 涉及核心机密的任务

举例：处理未公开的财务数据、撰写并购方案、准备上市材料

为什么不划算：风险太大，一旦泄露后果严重。

判断标准：如果数据泄露会导致法律问题或重大损失，别用 Agent。

● 5. 需要人际关系的任务

举例：安慰情绪低落的同事、处理团队冲突、给老板提建议

为什么不划算：这些任务的核心是"人与人的连接"，Agent 做不到。

判断标准：如果任务的核心是"关系"而不是"信息"，别用 Agent。

● 6. 验收成本极高的任务

举例：法律合同审查、医疗诊断辅助、安全审计

为什么不划算：Agent 做完后，你需要花同样多的时间验收，等于做了两遍。

判断标准：如果验收时间 自己做的的时间，别用 Agent。

● 一个总结

- 1 适合 Agent 的任务：重复 + 规则明确 + 信息密集 + 验收简单
- 2 不适合 Agent 的任务：一次性 + 需要创意 + 需要判断 + 验收复杂

本章小结

- * Agent 幻觉比 ChatBot 更危险，因为它会直接执行错误操作
- * 永远不要完全信任 Agent，保持验收习惯（抽样、对比、交叉验证）
- * 算清三本账：token 成本、时间成本、决策质量
- * 6 类任务不划算：一次性简单任务、高度创意、频繁决策、核心机密、人际关系、验收成本极高

附录 A：30 条 Brief 模板（按职能分类）

● 市场部（6 条）

模板 1：竞品动态周报

- 1 【背景】我是市场部竞品分析师，每周一需要提交竞品动态周报。
- 2 【任务】
- 3 目标：生成本周竞品动态周报
- 4 约束：只看 {{竞品列表}}，每家至少 3 条动态，按时间倒序
- 5 素材：从 {{数据源}} 抓取 {{日期范围}} 的新闻
- 6 验收：Word 文档，含竞品名称、动态标题、来源链接、一句话总结、影响评估
- 7 【交付】文件名 competitor_weekly_{{日期}}.docx，保存到 desktop/reports/

模板 2：行业调研报告

- 1 【背景】需要了解 {{行业}} 的最新趋势和关键玩家。
- 2 【任务】
- 3 目标：生成行业调研报告
- 4 约束：覆盖市场规模、增长率、主要玩家、技术趋势、政策环境
- 5 素材：从 {{数据源}} 搜索 {{关键词}}，时间范围 {{日期范围}}
- 6 验收：Word 文档，含行业概览、竞争格局、趋势分析、机会与风险
- 7 【交付】文件名 industry_report_{{行业}}_{{日期}}.docx

模板 3：活动复盘报告

- 1 【背景】刚结束 {{活动名称}}，需要复盘效果。
- 2 【任务】
- 3 目标：生成活动复盘报告
- 4 约束：对比目标 vs 实际，分析成功因素和改进点
- 5 素材：活动数据 {{数据文件}}，包含参与人数、转化率、费用等
- 6 验收：PPT，含活动概览、数据对比、亮点、改进建议
- 7 【交付】文件名 event_review_{{活动名称}}_{{日期}}.pptx

模板 4：社媒舆情监控

- 1 【背景】需要监控品牌在社交媒体上的提及和情感。
- 2 【任务】
- 3 目标：生成本周品牌舆情报告
- 4 约束：覆盖微博、小红书、知乎，区分正面/中性/负面
- 5 素材：搜索品牌名 {{品牌名}} + 竞品名
- 6 验收：Excel，含平台、内容摘要、情感标注、热度、建议回应
- 7 【交付】文件名 sentiment_weekly_{{日期}}.xlsx

模板 5：内容日历规划

- 1 【背景】需要规划下月的内容发布计划。
- 2 【任务】
- 3 目标：生成下月内容日历
- 4 约束：每周至少 3 篇，覆盖 {{平台列表}}，结合热点和产品节奏
- 5 素材：产品路线图 {{文件}}，行业热点日历
- 6 验收：Excel，含日期、平台、主题、关键词、负责人
- 7 【交付】文件名 content_calendar_{{月份}}.xlsx

模板 6：竞品功能对比

- 1 【背景】需要对比我们和竞品的功能差异。
- 2 【任务】
- 3 目标：生成竞品功能对比表
- 4 约束：对比维度 {{维度列表}}，每个维度给出评分（1-5）
- 5 素材：竞品官网、产品文档、用户评价
- 6 验收：Excel，含功能矩阵 + 差异分析 + 我们的优劣势总结
- 7 【交付】文件名 feature_comparison_{{日期}}.xlsx

● 销售部（6 条）

模板 7：销售线索打分

- 1 【背景】我是 SDR，需要筛选和打分今天的销售线索。
- 2 【任务】
- 3 目标：筛选并打分销售线索
- 4 约束：A 级（{{A级标准}}）、B 级（{{B级标准}}）、C 级（其他）
- 5 素材：线索文件 {{文件路径}}
- 6 验收：Excel，含评分、理由、跟进话术、建议跟进时间
- 7 【交付】文件名 leads_scored_{{日期}}.xlsx，按评分排序

模板 8：客户背景调研

- 1 【背景】即将拜访客户 {{客户名称}}，需要了解背景。
- 2 【任务】
- 3 目标：生成客户背景调研报告
- 4 约束：覆盖公司概况、业务模式、近期动态、可能的痛点
- 5 素材：从官网、新闻、招聘信息搜索
- 6 验收：Word，1-2 页，含公司概况、业务分析、合作切入点
- 7 【交付】文件名 client_brief_{{客户名称}}_{{日期}}.docx

模板 9：报价方案生成

- 1 【背景】客户 {{客户名称}} 需要报价，规模 {{用户数}} 人。
- 2 【任务】
- 3 目标：生成报价方案
- 4 约束：基于标准价格表 {{价格表文件}}，根据规模给出折扣建议
- 5 素材：价格表、客户需求清单 {{需求文件}}
- 6 验收：Word，含方案概述、功能清单、价格明细、付款方式
- 7 【交付】文件名 quotation_{{客户名称}}_{{日期}}.docx

模板 10：销售周报

- 1 【背景】每周五需要提交销售周报给主管。
- 2 【任务】
- 3 目标：生成本周销售周报
- 4 约束：重点写新增线索、跟进进展、成交情况、下周计划
- 5 素材：CRM 数据 {{文件}} 或本周跟进记录 {{文件}}
- 6 验收：Word，含本周数据、重点客户进展、下周目标
- 7 【交付】文件名 sales_weekly_{{日期}}.docx

模板 11：竞品话术库

- 1 【背景】客户经常问“你们和 {{竞品}} 有什么区别”。
- 2 【任务】
- 3 目标：生成竞品对比话术
- 4 约束：每个竞品 3-5 个常见问题 + 应对话术，语气专业但不贬低对手
- 5 素材：竞品官网、用户评价、我们的优势清单 {{文件}}
- 6 验收：Word，按竞品分章节，每章含常见问题和话术
- 7 【交付】文件名 competitive_playbook_{{日期}}.docx

模板 12：客户成功案例

- 1 【背景】需要整理客户 {{客户名称}} 的成功案例用于营销。
- 2 【任务】
- 3 目标：生成客户成功案例文档
- 4 约束：结构为“挑战 → 方案 → 成果”，含具体数据
- 5 素材：客户访谈记录 {{文件}}、使用数据 {{文件}}
- 6 验收：Word，1-2 页，含客户背景、挑战、我们的方案、量化成果
- 7 【交付】文件名 case_study_{{客户名称}}.docx

● 财务部（6 条）

模板 13：月度对账

- 1 【背景】每月初核对上月银行流水和财务系统记录。
- 2 【任务】
- 3 目标：核对 {{月份}} 的银行流水和财务系统记录
- 4 约束：金额差异 > 0.01 元标注异常，排除内部转账
- 5 素材：银行流水 {{文件}}，财务系统记录 {{文件}}
- 6 验收：Excel，分两个 Sheet（一致/异常），异常含类型和建议处理方式
- 7 【交付】文件名 reconciliation_{{月份}}.xlsx

模板 14：发票分类整理

- 1 【背景】收到一批发票需要分类整理。
- 2 【任务】
- 3 目标：将发票按供应商和类型分类
- 4 约束：分类维度为供应商、费用类型（差旅/办公/服务/其他）、金额区间
- 5 素材：发票文件夹 {{路径}}（PDF 格式）
- 6 验收：Excel，含发票号、供应商、金额、类型、日期
- 7 【交付】文件名 invoice_summary_{{月份}}.xlsx

模板 15：费用报销审核

- 1 【背景】需要审核本月的费用报销单。
- 2 【任务】
- 3 目标：审核报销单是否符合公司政策
- 4 约束：审核规则 {{规则文件}}，标注不合规项
- 5 素材：报销单 {{文件夹路径}}
- 6 验收：Excel，含报销人、金额、是否合规、不合规原因
- 7 【交付】文件名 expense_audit_{{月份}}.xlsx

模板 16：预算执行分析

- 1 【背景】需要分析本月预算执行情况。
- 2 【任务】
- 3 目标：对比预算 vs 实际支出，标注超支项
- 4 约束：超支 > 10% 标注"需关注"，超支 > 20% 标注"严重超支"
- 5 素材：预算表 {{文件}}，实际支出 {{文件}}
- 6 验收：Excel，含科目、预算、实际、差异、差异率、标注
- 7 【交付】文件名 budget_analysis_{{月份}}.xlsx

模板 17：应收账款跟踪

- 1 【背景】需要跟踪应收账款的回款情况。
- 2 【任务】
- 3 目标：生成应收账款跟踪表
- 4 约束：逾期 > 30 天标注"催收"，逾期 > 90 天标注"风险"
- 5 素材：应收台账 {{文件}}，回款记录 {{文件}}
- 6 验收：Excel，含客户、金额、到期日、逾期天数、状态、建议动作
- 7 【交付】文件名 ar_tracking_{{月份}}.xlsx

模板 18：财务月报

- 1 【背景】每月需要生成财务月报给管理层。
- 2 【任务】
- 3 目标：生成 {{月份}} 财务月报
- 4 约束：含收入、支出、利润、同比环比、关键指标
- 5 素材：财务数据 {{文件}}，上月/去年同期数据 {{文件}}
- 6 验收：PPT，含关键数据、趋势图、异常说明、下月预测
- 7 【交付】文件名 finance_monthly_{{月份}}.pptx

● 产品部（6 条）

模板 19：用户反馈主题洞察

- 1 【背景】每周整理用户反馈，提取核心诉求。
- 2 【任务】
- 3 目标：从用户反馈中提取主题并生成洞察报告
- 4 约束：分类为功能需求/性能/UI/Bug/其他，按提及次数排优先级
- 5 素材：反馈文件 {{文件路径}}
- 6 验收：Excel，含主题、分类、提及次数、优先级、典型反馈、建议
- 7 【交付】文件名 feedback_insights_{{周次}}.xlsx

模板 20：需求文档（PRD）框架

- 1 【背景】需要为 {{功能名称}} 写需求文档。
- 2 【任务】
- 3 目标：生成 PRD 框架
- 4 约束：含背景、目标、用户故事、功能描述、验收标准、排期建议
- 5 素材：产品路线图 {{文件}}，用户反馈 {{文件}}，竞品参考 {{链接}}
- 6 验收：Word，结构完整，可直接交给研发评审
- 7 【交付】文件名 PRD_{{功能名称}}_{{日期}}.docx

模板 21：版本发布说明

- 1 【背景】即将发布 v{{版本号}}, 需要写发布说明。
- 2 【任务】
- 3 目标: 生成版本发布说明 (面向用户)
- 4 约束: 语言通俗易懂, 突出用户价值, 不写技术细节
- 5 素材: 开发完成的功能清单 {{文件}}, Bug 修复清单 {{文件}}
- 6 验收: Markdown, 含新功能、改进、修复, 每条 1-2 句话
- 7 【交付】文件名 release_notes_v{{版本号}}.md

模板 22: 用户调研问卷设计

- 1 【背景】需要调研用户对 {{功能/主题}} 的看法。
- 2 【任务】
- 3 目标: 设计用户调研问卷
- 4 约束: 10-15 题, 含单选/多选/开放题, 预计填写时间 5 分钟
- 5 素材: 调研目标 {{描述}}, 目标用户画像 {{描述}}
- 6 验收: Word, 含问卷题目、选项、逻辑跳转说明
- 7 【交付】文件名 survey_{{主题}}_{{日期}}.docx

模板 23: 竞品功能拆解

- 1 【背景】需要深度拆解竞品 {{竞品名}} 的某个功能。
- 2 【任务】
- 3 目标: 拆解竞品功能的设计逻辑
- 4 约束: 覆盖用户流程、交互设计、数据结构、商业逻辑
- 5 素材: 竞品产品截图 {{文件夹}}, 官方文档 {{链接}}
- 6 验收: Word, 含功能概述、用户流程图、设计亮点、可借鉴点
- 7 【交付】文件名 teardown_{{竞品名}}_{{功能}}_{{日期}}.docx

模板 24: 数据埋点方案

- 1 【背景】新功能 {{功能名}} 上线, 需要设计数据埋点。
- 2 【任务】
- 3 目标: 生成数据埋点方案
- 4 约束: 覆盖关键路径的每个节点, 含事件名、属性、触发条件
- 5 素材: 功能 PRD {{文件}}, 现有埋点规范 {{文件}}
- 6 验收: Excel, 含事件名、触发条件、属性列表、数据类型
- 7 【交付】文件名 tracking_plan_{{功能名}}_{{日期}}.xlsx

● 研发部 (6 条)

模板 25: 技术选型调研

- 1 【背景】团队需要引入 {{技术类型}}，候选方案 {{候选列表}}。
- 2 【任务】
- 3 目标：对比候选方案，生成技术选型报告
- 4 约束：对比维度为性能、成本、社区、学习曲线、适用场景
- 5 素材：官方文档、GitHub、技术博客
- 6 验收：Word，含对比表、优缺点、推荐方案和理由
- 7 【交付】文件名 tech_selection_{{技术类型}}_{{日期}}.docx

模板 26：代码审查清单

- 1 【背景】需要审查 {{项目/模块}} 的代码质量。
- 2 【任务】
- 3 目标：生成代码审查报告
- 4 约束：检查维度为安全性、性能、可维护性、测试覆盖
- 5 素材：代码仓库 {{路径}}，审查范围 {{描述}}
- 6 验收：Markdown，含问题清单、严重程度、修复建议
- 7 【交付】文件名 code_review_{{模块}}_{{日期}}.md

模板 27：故障复盘报告

- 1 【背景】{{日期}} 发生了 {{故障描述}}，需要复盘。
- 2 【任务】
- 3 目标：生成故障复盘报告（Postmortem）
- 4 约束：结构为时间线、根因分析、影响范围、改进措施
- 5 素材：监控日志 {{文件}}，处理记录 {{文件}}
- 6 验收：Word，含时间线、5-Why 分析、改进 Action Items
- 7 【交付】文件名 postmortem_{{日期}}_{{故障简述}}.docx

模板 28：API 文档生成

- 1 【背景】需要为 {{模块}} 的 API 生成文档。
- 2 【任务】
- 3 目标：生成 API 接口文档
- 4 约束：含接口路径、方法、参数、返回值、错误码、示例
- 5 素材：代码文件 {{路径}}，现有文档 {{文件}}
- 6 验收：Markdown，符合 OpenAPI 规范，可直接发布
- 7 【交付】文件名 api_docs_{{模块}}_{{日期}}.md

模板 29：性能测试报告

- 1 【背景】需要对 {{系统/模块}} 进行性能测试分析。
- 2 【任务】
- 3 目标：分析性能测试结果，生成报告
- 4 约束：关注 QPS、延迟（P50/P95/P99）、错误率、资源使用
- 5 素材：测试结果 {{文件}}，基准数据 {{文件}}
- 6 验收：Word，含测试概况、关键指标、瓶颈分析、优化建议
- 7 【交付】文件名 perf_report_{{模块}}_{{日期}}.docx

模板 30：技术方案设计

- 1 【背景】需要为 {{需求描述}} 设计技术方案。
- 2 【任务】
- 3 目标：生成技术方案文档
- 4 约束：含架构设计、接口设计、数据模型、风险评估、排期
- 5 素材：需求文档 {{文件}}，现有架构图 {{文件}}
- 6 验收：Word，含架构图、时序图、接口定义、里程碑
- 7 【交付】文件名 tech_design_{{功能}}_{{日期}}.docx

附录 B：MCP 生态地图与接入清单

B.1 MCP 生态概览

截至 2026 年 5 月，MCP 生态已有 500+ 插件。以下是按类别整理的常用插件清单。

办公协作类

插件名称	功能	接入难度	适合场景
企业微信 MCP	发消息、拉群、读取聊天记录	低	通知、协作
飞书 MCP	文档、日历、审批	低	字节系企业
钉钉 MCP	消息、审批、考勤	低	阿里系企业
Slack MCP	频道消息、文件共享	低	外企
Notion MCP	数据库、页面、看板	低	知识管理

数据分析类

插件名称	功能	接入难度	适合场景
------	----	------	------

1. 在 WorkBuddy 设置 → MCP 市场 → 搜索插件名称
2. 点击"安装"
3. 按提示授权（通常需要对应平台的 API Key 或 OAuth 登录）
4. 测试连接（发送一条测试指令）
5. 在 Brief 中直接调用

B.3 自研 MCP 开发指南（简要）

如果市场上没有现成插件，可以用 MiniClaw 框架自研：

1. 安装 MiniClaw: `pip install miniclaw`
2. 定义工具描述（JSON Schema）
3. 实现工具逻辑（Python 函数）
4. 注册到 WorkBuddy
5. 测试和发布

详细开发文档参考：MiniClaw GitHub 仓库。

附录 C：常见故障诊断树

C.1 Agent 不执行任务

- 1 Agent 不执行任务
- 2 | 是否理解了任务？
 - 3 | | 没理解 → Brief 不够清晰，重写（参考第 3 章）
 - 4 | | 理解了但不动 → 继续排查
- 5 | 是否有权限？
 - 6 | | 文件权限不足 → 检查沙箱授权范围
 - 7 | | API 未配置 → 检查 MCP 设置
 - 8 | | 有权限 → 继续排查
- 9 | 是否有资源？
 - 10 | | 网络不通 → 检查网络连接
 - 11 | | 文件不存在 → 检查文件路径
 - 12 | | 有资源 → 联系技术支持
- 13 | 其他
 - 14 | | 重启 WorkBuddy 后重试

C.2 Agent 执行结果不对

- 1 执行结果不对
- 2 | 方向错了 (做的不是你要的)
- 3 | | Brief 目标不清晰 → 重写目标部分
- 4 | | Agent 理解偏差 → 加"先列计划"确认
- 5 | | 素材给错了 → 检查文件路径和内容
- 6 | 方向对但质量差
- 7 | | 约束不够 → 加更多约束条件
- 8 | | 素材不足 → 补充背景信息
- 9 | | 验收标准模糊 → 明确"什么算好"
- 10 | 部分对部分错
- 11 | | 找到出错的步骤 → 只纠偏那一步
- 12 | | 整体逻辑有问题 → 重新设计 Brief

C.3 Agent 卡住不动

- 1 Agent 卡住不动
- 2 | 在哪一步卡住?
- 3 | | 网页抓取 → 网站可能有反爬, 换数据源
- 4 | | 文件处理 → 文件可能损坏或太大, 检查文件
- 5 | | API 调用 → API 可能超时, 检查网络
- 6 | | 不确定 → 看任务树, 找到最后一个 的步骤
- 7 | 卡了多久?
- 8 | | < 1 分钟 → 等一等, 可能在处理
- 9 | | 1-5 分钟 → 可能有问题, 准备中断
- 10 | | > 5 分钟 → 中断, 检查原因
- 11 | 中断后怎么办?
- 12 | | 修复问题后重试
- 13 | | 跳过这一步, 继续后面的
- 14 | | 换一种方式完成这个任务

C.4 Agent 消耗 token 过多

- 1 Token 消耗过多
 - 2 | Brief 太长或太模糊?
 - 3 | | 太长 → 精简，只保留必要信息
 - 4 | | 太模糊 → Agent 在试错，写清楚约束
 - 5 | 任务太大?
 - 6 | | 拆分成多个小任务
 - 7 | | 用模板减少重复描述
 - 8 | Agent 在循环重试?
 - 9 | | 某一步一直失败 → 中断，手动解决
 - 10 | | 设置最大重试次数
 - 11 | 数据量太大?
 - 12 | | 减少输入数据量（采样）
 - 13 | | 分批处理

C.5 安全相关问题

- 1 安全问题
 - 2 | Agent 访问了不该访问的文件
 - 3 | | 检查沙箱授权范围
 - 4 | | 缩小授权范围
 - 5 | | 查看审计日志确认影响
 - 6 | Agent 执行了不该执行的操作
 - 7 | | 检查是否开启了"关键操作确认"
 - 8 | | 在 Brief 里加"不可逆操作必须先确认"
 - 9 | | 查看操作日志，评估影响
 - 10 | 怀疑数据泄露
 - 11 | | 检查 Agent 是否调用了外部 API
 - 12 | | 检查网络日志
 - 13 | | 联系安全团队

附录 D：从入门到熟练的 30 天学习路径

第 1 周：建立认知（Day 1-7）

天数	学习内容	实践任务	预计时间
Day 1	读第 1 章，理解 Agent vs ChatBot	用 WorkBuddy 做一个简单文件整理	30 分钟
Day 2	读第 2 章，理解管理者思维	用四件套写一个 Brief	30 分钟

Day 3	读第 3 章，学习 Brief 写法	用三段式模板写一个完整 Brief	45 分钟
Day 4	读第 4 章，理解执行链路	让 Agent "先列计划"，观察任务树	30 分钟
Day 5	读第 5 章，理解能力边界	测试沙箱边界（故意给一个越权指令）	30 分钟
Day 6	复习第 1-5 章	独立完成一个组合任务（不看教程）	45 分钟
Day 7	总结本周收获	写下 3 个"我学到的"和 3 个"我还不确定的"	15 分钟

● 第 2 周：场景实战（Day 8-14）

天数	学习内容	实践任务	预计时间
Day 8	读第 8.1 节	用竞品周报模板做一次（可以用假数据）	45 分钟
Day 9	读第 8.2 节	用销售线索打分模板做一次	45 分钟
Day 10	读第 8.3 节	用月度对账模板做一次	45 分钟
Day 11	读第 8.4 节	用用户反馈洞察模板做一次	45 分钟
Day 12	读第 8.5 节	用技术选型调研模板做一次	45 分钟
Day 13	选一个和你工作最相关的场景	用真实数据做一次，记录问题	1 小时
Day 14	总结本周收获	把做得最好的 Brief 保存为模板	30 分钟

● 第 3 周：模板化与复用（Day 15-21）

天数	学习内容	实践任务	预计时间
Day 15	读第 6 章（技能包）	浏览预置技能包，找到 3 个和你工作相关的	30 分钟
Day 16	读第 7 章（工作流）	识别你工作中的"可复用任务"	30 分钟
Day 17	模板化练习	把你最常做的任务写成模板	45 分钟

Day 18	模板化练习	再写 2 个模板	45 分钟
Day 19	组合技能包	尝试串联 2 个技能包完成一个任务	45 分钟
Day 20	建立 Brief 库	整理你的所有模板，按分类存放	30 分钟
Day 21	总结本周收获	你的 Brief 库应该有 5+ 个模板了	15 分钟

第 4 周：进阶与自动化（Day 22-30）

天数	学习内容	实践任务	预计时间
Day 22	读第 9 章（协作）	把你的模板分享给 1 个同事	30 分钟
Day 23	读第 10 章（判断力）	列出你工作中“不适合用 Agent”的任务	30 分钟
Day 24	自动化入门	给一个重复任务配置定时触发器	45 分钟
Day 25	自动化进阶	配置异常告警（失败时通知你）	45 分钟
Day 26	MCP 探索	安装 1 个第三方 MCP 插件	30 分钟
Day 27	综合实战	设计一个多步骤工作流（串联 3+ 个任务）	1 小时
Day 28	综合实战	执行并优化这个工作流	1 小时
Day 29	复盘	对比 Day 1 和 Day 29 的效率差异	30 分钟
Day 30	规划下一步	列出未来 3 个月要自动化的任务清单	30 分钟

30 天后你应该达到的水平

- * [完成] 能独立写出高质量 Brief（不需要参考模板）
- * [完成] 有 10+ 个可复用模板
- * [完成] 至少 1 个任务实现了自动化
- * [完成] 能判断什么任务适合/不适合用 Agent
- * [完成] 每周节省 5+ 小时的重复劳动

持续精进的方向

方向	内容	适合人群
深度模板化	把所有重复任务都模板化	所有人
自动化	配置更多触发器和工作流	效率控
MCP 开发	自研插件对接企业系统	技术人员
团队推广	建立团队 Brief 库和规范	管理者

结语

Agent 不是魔法，而是工具。

它不会让你变得更聪明，但会让你的时间更值钱。

把重复的、规则明确的、信息密集的工作交给 Agent，把你的时间留给需要创意、判断和人际连接的工作。

这才是 AI 时代知识工作者的正确姿势：不是被 AI 替代，而是用 AI 放大。

本文档基于 WorkBuddy 2026 年 5 月版本编写。产品功能可能随版本更新而变化，请以官方文档为准。